



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo fin de Grado

Grado en Ingeniería Informática (Ingeniería del Software)

BargAIn: Sistema inteligente de optimización de la cesta de la compra

**Realizado por
Nicolás Parrilla Geniz**

**Dirigido por
Juan Vicente Gutiérrez Santacreu**

**Departamento
Matemática Aplicada I**

Sevilla, Junio 2026

Resumen

BargAIn es una plataforma de compra inteligente orientada a optimizar la cesta de la compra del consumidor combinando tres variables de decisión: precio, distancia y tiempo. El proyecto desarrolla una solución full-stack con backend en Django/DRF, base de datos PostgreSQL + PostGIS, cliente móvil en React Native (Expo), y companion web para el portal de comercios.

La propuesta integra cuatro capacidades diferenciales en una única aplicación: (1) comparación multi-tienda de precios, (2) optimización multicriterio de ruta con OR-Tools, (3) digitalización de listas y tickets mediante OCR con Google Cloud Vision, y (4) asistente conversacional basado en LLM (Google Gemini) vía backend proxy.

Desde la perspectiva de ingeniería del software, el trabajo cubre el ciclo completo: análisis de requisitos, diseño arquitectónico, implementación modular, pruebas unitarias/integración/E2E, despliegue en staging y documentación técnica. La arquitectura se organiza en dominios desacoplados (users, products, stores, prices, shopping_lists, optimizer, ocr, assistant, business, notifications), con tareas asíncronas en Celery y Redis para procesos pesados.

Como resultado, se obtiene un prototipo funcional validado en entorno de desarrollo y staging, con cobertura de pruebas en módulos críticos, flujos E2E automatizados para el web companion, y verificaciones manuales en flujos móviles dependientes de GPS/cámara. La memoria documenta además limitaciones, decisiones arquitectónicas y líneas de evolución para convertir BargAIn en un producto escalable.

Abstract

BargAI is an intelligent shopping platform focused on optimizing grocery baskets by combining three decision variables: price, distance, and time. The project delivers a full-stack solution with a Django/DRF backend, PostgreSQL + PostGIS geospatial storage, React Native (Expo) mobile client, and a web companion for small businesses.

The proposal integrates four key capabilities in a single system: (1) multi-store price comparison, (2) multicriteria route optimization with OR-Tools, (3) OCR-based list/ticket digitization using Google Cloud Vision, and (4) a conversational assistant powered by an LLM (Google Gemini) through a backend proxy.

From a software engineering standpoint, the work covers the whole lifecycle: requirements analysis, architecture design, modular implementation, unit/integration/E2E testing, staging deployment, and technical documentation. The architecture is organized into decoupled domains, with asynchronous processing delegated to Celery workers over Redis.

The final outcome is a functional prototype validated through automated and manual tests, with clear traceability of technical decisions, current limitations, and future evolution paths toward production readiness.

Agradecimientos

A mi tutor, Juan Vicente Gutiérrez Santacreu, por su orientación constante, su criterio técnico y su apoyo durante todo el desarrollo del TFG.

A mi familia y personas cercanas, por su paciencia y motivación en las fases de mayor carga de trabajo.

A la comunidad de software libre y a la documentación técnica de las herramientas utilizadas, que han sido clave para avanzar con rigor en el diseño, la implementación y la validación de BargAIn.

Índice general

Índice general	IV
Índice de cuadros	VII
Índice de figuras	VIII
1 Introducción y objetivos del proyecto	1
1.1 Contexto y motivación	1
1.2 Problema abordado	1
1.3 Objetivo principal	1
1.4 Objetivos específicos	1
1.5 Alcance del proyecto	2
1.6 Tecnologías clave	2
1.7 Estructura de la memoria	3
2 Análisis de antecedentes y aportación realizada	4
2.1 Contexto del sector	4
2.2 Estado del arte en comparación de precios	4
2.3 Tecnologías relevantes para BargAIIn	4
2.3.1 Geoespacial y rutas	4
2.3.2 Ingesta de datos de precio	4
2.3.3 OCR y asistente	5
2.4 Aportación diferencial de BargAIIn	5
3 Comparación con otras alternativas	6
3.1 Competidores considerados	6
3.2 Matriz funcional resumida	6
3.3 Brechas estratégicas detectadas	6
3.4 Posicionamiento de BargAIIn	7
3.5 Riesgos competitivos	7
4 Análisis temporal y costes de desarrollo	8
4.1 Planificación temporal	8
4.2 Metodología de trabajo	8
4.3 Estimación de costes	9
4.4 Riesgos y mitigaciones	9
4.5 Resultado de planificación	9

5	Análisis de requisitos	10
5.1	Actores del sistema	10
5.2	Requisitos de información	10
5.3	Requisitos funcionales	10
5.4	Criterios de aceptación del producto	11
5.5	Requisitos no funcionales	11
5.6	Reglas de negocio	12
5.7	Diagramas de casos de uso	12
5.8	Trazabilidad	12
6	Diseño e Implementación	15
6.1	Arquitectura del Sistema	15
6.1.1	Visión general	15
6.1.2	Patrón de organización interna del backend	15
6.1.3	Comunicación entre componentes	16
6.2	Modelo de Datos	16
6.2.1	Módulo de usuarios	16
6.2.2	Módulo de productos	17
6.2.3	Módulo de tiendas	17
6.2.4	Módulo de precios	17
6.2.5	Módulo del optimizador	18
6.2.6	Índices clave	18
6.2.7	Modelo entidad-relación	18
6.2.8	Diagrama de clases	18
6.3	Diseño de la API REST	18
6.3.1	Estructura general	18
6.3.2	Endpoints principales	20
6.3.3	Autenticación y autorización	20
6.4	Implementación del Backend	20
6.4.1	Módulo de notificaciones	20
6.4.2	Módulo de portal business	21
6.5	El Algoritmo de Optimización de Rutas	21
6.5.1	Formulación del problema	21
6.5.2	Función de scoring multicriterio	21
6.5.3	Pipeline de ejecución	21
6.5.4	Diagrama de secuencia del optimizador	22
6.5.5	Integración con OR-Tools	22
6.5.6	Complejidad y rendimiento	22
6.6	Módulo de Web Scraping	24
6.7	Módulo OCR	24
6.7.1	Diagrama de secuencia OCR	24
6.8	Integración del Asistente LLM	24
6.9	Infraestructura y Despliegue	26
6.9.1	Contenedores Docker	26
6.9.2	Pipeline CI/CD	26
6.9.3	Variables de entorno	26
6.10	Diseño de la Interfaz de Usuario	26
6.10.1	Sistema de diseño	26
6.10.2	Pantallas principales	27
6.10.3	Adaptación a uso web (responsive y conveniencias de escritorio)	27

7	Manual de Usuario	29
7.1	Introducción	29
7.2	Requisitos de uso	29
7.3	Registro y acceso al sistema	29
7.4	Pantalla principal	30
7.5	Gestión de listas de la compra	32
7.6	Catálogo y comparativa de precios	34
7.7	Mapa de tiendas	36
7.8	Alertas de precio y notificaciones	38
7.9	Perfil y preferencias del usuario	38
7.10	Ruta de compra optimizada	41
7.11	Reconocimiento de tickets y listas (OCR)	42
7.12	Asistente conversacional	42
7.13	Lista de comprobación en pantalla de bloqueo (iOS)	43
7.14	La aplicación en el navegador de escritorio	44
7.15	Portal Business para PYMEs	48
7.16	Resolución de incidencias comunes	52
8	Pruebas y Resultados	53
8.1	Objetivo	53
8.2	Estrategia de testing	53
8.3	Entorno de ejecución	53
8.4	Suite de tests backend	54
8.4.1	Tests de integración	54
8.4.2	Cobertura	54
8.5	Tests E2E con Playwright	54
8.5.1	Configuración Playwright	55
8.6	UAT manual en dispositivo móvil	55
8.7	Validación de Requisitos No Funcionales	55
8.7.1	RNF-002: Disponibilidad $\geq 99\%$ en staging	55
8.7.2	RNF-004: Usabilidad WCAG 2.1 AA y máximo 3 taps	56
8.7.3	RNF-005: Escalabilidad para 10.000 usuarios	56
8.8	Resumen de resultados	56
8.9	Conclusión	56
9	Conclusiones	58
9.1	Grado de cumplimiento	58
9.2	Aportaciones principales del trabajo	58
9.3	Limitaciones	59
9.4	Lecciones aprendidas	59
9.5	Trabajo futuro	60
9.6	Cierre	60
	Referencias	61

Índice de cuadros

3.1	Comparativa de capacidades clave	6
4.1	Resumen por fases del proyecto	8
4.2	Coste estimado del desarrollo	9
5.1	Agrupación de requisitos funcionales	11
6.1	Índices principales de la base de datos	18
6.2	Endpoints de autenticación y usuarios	20
6.3	Endpoints del optimizador, OCR y asistente	20
6.4	Tiempos estimados de ejecución del optimizador	22
6.5	Paleta de colores del sistema de diseño	27
8.1	Suite de tests de integración backend	54
8.2	Suite de tests E2E con Playwright	55
8.3	Resultados UAT en dispositivo móvil	55
8.4	Resumen de resultados de la estrategia de testing	56

Índice de figuras

5.1	Casos de uso del Consumidor	13
5.2	Casos de uso del Comercio y Administrador	14
6.1	Arquitectura por capas de BargAIn	16
6.2	Diagrama entidad-relación de BargAIn	19
6.3	Diagrama de clases del dominio	19
6.4	Diagrama de secuencia del proceso de optimización de ruta	23
6.5	Diagrama de secuencia del flujo OCR	25
7.1	Acceso al sistema: inicio de sesión (izquierda) y registro de un nuevo usuario (derecha)	30
7.2	Pantalla principal de la aplicación móvil, con notificaciones, listas activas, tiendas cercanas y alertas de precio	31
7.3	Gestión de listas: colección de listas del usuario (izquierda) y detalle de una lista con controles de cantidad y verificación (derecha)	32
7.4	Plantillas de lista: instanciación de compras recurrentes a partir de una plantilla guardada	33
7.5	Catálogo de productos con filtros y añadido rápido (izquierda) y comparativa de precios por tienda de un producto (derecha)	34
7.6	Propuesta de alta de un nuevo producto en el catálogo, sujeta a aprobación por el administrador	35
7.7	Mapa de tiendas con marcadores por cadena y panel de establecimientos (izquierda) y perfil de tienda con sus productos y precios (derecha)	36
7.8	Tiendas favoritas del usuario, con acceso directo a su perfil	37
7.9	Bandeja de notificaciones con avisos de precio, promociones y listas compartidas (izquierda) y gestión de alertas de precio con su objetivo (derecha)	38
7.10	Perfil del usuario (izquierda) y configuración del optimizador con radio, paradas y pesos del <i>scoring</i> multicriterio (derecha)	39
7.11	Operaciones sobre la cuenta: edición del perfil (izquierda) y cambio de contraseña (derecha)	40
7.12	Ruta de compra optimizada: parámetros, resultado agregado, desglose por parada y resolución de ambigüedades semánticas con recálculo	41
7.13	Asistente OCR: captura del ticket o lista (izquierda) y revisión de los ítems reconocidos con su nivel de confianza (derecha)	42
7.14	Asistente conversacional con sugerencias de inicio y campo de consulta; las respuestas con productos incluyen fichas de añadido directo a la lista	43
7.15	Versión de escritorio: pantalla principal con accesos horizontales (izquierda) y catálogo en rejilla multicolumna con soporte de teclado (derecha)	44
7.16	Comparativa de precios con cabecera fija y exportación CSV (izquierda) y detalle de lista con reordenado por arrastre y exportación (derecha)	44

7.17 Colecciones en rejilla responsiva: listas de la compra (izquierda) y plantillas (derecha)	45
7.18 Mapa con panel lateral de tiendas (izquierda) y perfil de tienda a dos columnas (derecha)	45
7.19 Tiendas favoritas en rejilla (izquierda) y asistente conversacional centrado con copia por mensaje (derecha)	46
7.20 Ruta optimizada con exportación del itinerario (izquierda) y módulo OCR adaptado al escritorio (derecha)	46
7.21 Bandeja de notificaciones (izquierda) y alertas de precio (derecha) en la versión de escritorio	47
7.22 Página de presentación del portal Business, con la propuesta de valor para el comercio local y el proceso de alta en tres pasos	48
7.23 Acceso al portal Business: inicio de sesión (izquierda) y registro de una cuenta de comercio (derecha)	48
7.24 <i>Onboarding</i> del comercio: alta de los datos legales y comerciales que un administrador verificará antes de activar la cuenta	49
7.25 Centro de operaciones del comercio: indicadores de actividad, salud de promociones y ajustes recientes de precio	50
7.26 Gestión del surtido: resumen de productos del comercio (izquierda) y carga masiva de catálogo mediante CSV con validación (derecha)	50
7.27 Gestión de precios: tabla editable con panel lateral de edición de precio base, unitario y oferta con caducidad	51
7.28 Promociones del comercio con analítica básica (izquierda) y perfil del negocio con umbral de alerta competitiva (derecha)	51
7.29 Panel de administración: verificación y aprobación de perfiles PYME y de propuestas de producto	52

CAPÍTULO 1

Introducción y objetivos del proyecto

1.1— Contexto y motivación

El consumidor español afronta la compra cotidiana en un contexto de fuerte variabilidad de precios entre cadenas, ausencia de una fuente unificada y alto coste temporal para comparar ofertas manualmente. Aunque el ahorro potencial entre supermercados es significativo, en la práctica ese ahorro se pierde cuando no se considera el coste de desplazamiento ni el tiempo de ruta.

BargAIn nace para resolver ese problema como una plataforma de compra inteligente que integra precio, distancia y tiempo en una única decisión. El sistema combina una app móvil para consumidores, un portal web para PYMEs, y un backend modular con capacidades geoespaciales, OCR y asistencia conversacional.

1.2— Problema abordado

Las soluciones actuales de comparación de supermercados suelen responder a la pregunta “qué tienda tiene el precio más bajo”, pero no a la pregunta completa “qué compra me conviene realmente según mi ubicación y mis preferencias”.

El problema de BargAIn se formula como una optimización multicriterio en la que intervienen:

- coste total de la cesta,
- distancia adicional de desplazamiento,
- tiempo total estimado de compra y ruta.

1.3— Objetivo principal

Desarrollar una aplicación móvil y web que optimice la cesta de la compra del usuario, calculando la combinación de supermercados y comercios locales que ofrece la mejor relación precio–distancia–tiempo según preferencias configurables.

1.4— Objetivos específicos

1. Implementar una API REST con Django y DRF para usuarios, productos, tiendas, precios, listas, optimización, OCR, asistente y portal business.

2. Integrar una capa geoespacial con PostgreSQL + PostGIS para consultas de proximidad y preparación de rutas.
3. Desarrollar un optimizador multicriterio con OR-Tools y una función de scoring configurable por el usuario.
4. Incorporar OCR con Google Cloud Vision para digitalizar tickets y listas, con validación y corrección manual en frontend.
5. Integrar un asistente LLM (Gemini vía backend proxy) con guardrails de dominio y contexto de compra.
6. Construir un portal para PYMEs con gestión de precios y promociones.
7. Asegurar calidad con tests unitarios, de integración y E2E, además de UAT manual para flujos nativos.
8. Preparar despliegue reproducible en staging con Render, Docker Compose y CI.

1.5— Alcance del proyecto

El alcance del TFG cubre el ciclo completo de ingeniería del software:

- análisis y requisitos,
- diseño arquitectónico y de datos,
- implementación backend y frontend,
- pruebas funcionales y no funcionales,
- despliegue en entorno staging,
- documentación técnica y memoria final.

Quedan fuera del alcance de esta versión: despliegue productivo global, acuerdos formales con todas las cadenas para APIs oficiales, y la versión final de Live Activities nativas en iOS (ADR-010).

1.6— Tecnologías clave

El sistema se apoya en un stack full-stack moderno y coherente:

- **Backend:** Django 5, DRF, Celery, Redis.
- **Datos:** PostgreSQL 16 + PostGIS 3.4.
- **Frontend:** React Native con Expo + companion web en React.
- **IA:** Google Cloud Vision API (OCR) y Gemini vía backend proxy.
- **Optimización:** OR-Tools + cálculo de distancias (OpenRouteService + fallback haversine).
- **Ops:** Docker, GitHub Actions, Render, Sentry.

1.7– Estructura de la memoria

La memoria se organiza para reflejar el recorrido completo del proyecto: contexto y objetivos, estado del arte, comparativa, planificación, requisitos, diseño e implementación, manual de usuario, pruebas y resultados, conclusiones y bibliografía.

Análisis de antecedentes y aportación realizada

2.1— Contexto del sector

El mercado de distribución alimentaria en España presenta una combinación de gran competencia entre cadenas, alta sensibilidad al precio y fuerte fragmentación de oferta. Durante los últimos años, la inflación en alimentación ha incrementado la necesidad de herramientas que permitan comprar mejor sin aumentar el tiempo invertido.

Aunque existen comparadores de precios consolidados, la mayor parte se centra en mostrar información de coste por tienda, sin integrar de forma nativa el impacto logístico de desplazarse a varios establecimientos.

2.2— Estado del arte en comparación de precios

La evolución observada puede resumirse en tres generaciones:

1. **Directorios de precios:** listados estáticos o semiactualizados, con poca cobertura y baja frecuencia de refresco.
2. **Comparadores multisupermercado:** soluciones como Soysuper, Findit u OCU Market, con mejor catálogo y funcionalidades de lista.
3. **Personalización con IA:** tendencia emergente en la que se introducen recomendaciones contextuales y asistencia conversacional.

En España, el salto completo hacia la tercera generación todavía es limitado, especialmente en el segmento de cesta alimentaria diaria.

2.3— Tecnologías relevantes para BargAIn

2.3.1. Geoespacial y rutas

PostGIS permite resolver consultas de proximidad con precisión y rendimiento, mientras que OR-Tools facilita modelar el problema de ordenación de paradas como una variante del TSP/VRP en instancias pequeñas (máximo 3–4 paradas en nuestro caso).

2.3.2. Ingesta de datos de precio

La combinación Scrapy + Playwright es adecuada para catálogos dinámicos de supermercados. Sin embargo, por robustez de producto, BargAIn no depende de una única fuente y combina:

- scraping programado,
- aportaciones de crowdsourcing,
- actualización manual de comercios verificados.

2.3.3. OCR y asistente

Para OCR se adopta Google Cloud Vision API por mejor rendimiento práctico en tickets y fotos reales. Para el asistente se integra Gemini mediante backend proxy, con guardrails para limitar respuestas al dominio de la compra.

2.4— Aportación diferencial de BargAIn

La aportación del proyecto se concreta en cuatro ejes:

- **Optimización multicriterio real** (precio + distancia + tiempo) en lugar de comparación aislada de precios.
- **Inclusión de comercio local** mediante portal business y actualización de precios/promociones.
- **Digitalización de lista** por OCR con validación de usuario.
- **Interacción conversacional** para consultas complejas de compra.

Estas capacidades, integradas en una sola plataforma, posicionan BargAIn en un espacio de mercado poco cubierto por herramientas actuales.

CAPÍTULO 3

Comparación con otras alternativas

3.1— Competidores considerados

Para el análisis competitivo se han considerado cinco soluciones de referencia en el mercado español de comparación de compra:

- Soysuper
- Findit
- OCU Market
- PreciRadar
- RadarPrice

Adicionalmente, se han valorado como competencia indirecta las apps nativas de cadenas y como sustituto no tecnológico la comparación manual con folletos.

3.2— Matriz funcional resumida

Cuadro 3.1: Comparativa de capacidades clave

CAPACIDAD	BARGAIN	SOYSUPER	OCU MARKET	RESTO
Comparación multi-tienda de cesta	Completa	Completa	Parcial	Parcial
Optimización de ruta (precio + distancia + tiempo)	Completa	No	No	No
Integración de PYMEs / comercio local	Completa	No	No	No
OCR de lista / ticket	Completa	No	No	No
Asistente conversacional IA	Completa	No	No	No
Histórico avanzado de precios	Parcial	Limitado	Limitado	Variable

3.3— Brechas estratégicas detectadas

El análisis identifica cuatro brechas de mercado que justifican la propuesta:

1. **Gap precio-ruta:** los comparadores muestran precios, pero no optimizan desplazamiento y tiempo.

2. **Comercio local invisible:** no existe cobertura estable de PYMEs en herramientas generalistas.
3. **Fricción de entrada:** la lista de compra sigue siendo manual en la mayoría de soluciones.
4. **Interacción limitada:** predominan filtros y búsquedas rígidas frente a consultas en lenguaje natural.

3.4– Posicionamiento de BargAIn

BargAIn se posiciona como un *optimizador inteligente de compra* y no solo como comparador. Su propuesta de valor combina ahorro económico, eficiencia logística y experiencia de usuario asistida por IA.

3.5– Riesgos competitivos

Los riesgos principales identificados son:

- reacción de competidores consolidados con mayor base de usuarios,
- dependencia de fuentes de datos de terceros para scraping,
- necesidad de mantener alta calidad de datos para sostener confianza.

Como mitigación, se adopta una estrategia de fuentes híbridas de datos, modularidad arquitectónica y pipeline de validación continua.

CAPÍTULO 4

Análisis temporal y costes de desarrollo

4.1— Planificación temporal

La planificación de BargAIn se definió con una dedicación planificada de 300 horas, distribuidas en fases funcionales y técnicas, que se amplió hasta 325 horas reales al incorporar el cierre documental y la preparación de la defensa. El proyecto se ejecutó de forma iterativa, manteniendo trazabilidad en `TASKS.md` y en el árbol `.planning/`.

Cuadro 4.1: Resumen por fases del proyecto

FASE	SEMANAS	HORAS EST.	ESTADO
F1 · Análisis y diseño	S1–S3	45	Completada
F2 · Infraestructura base	S3–S5	30	Completada
F3 · Desarrollo core backend	S5–S12	95	Completada
F4 · Desarrollo frontend	S8–S16	70	Completada
F5 · IA, optimizador y scraping	S10–S17	35	Completada
F6 · Portal business y app móvil	S15–S18	25	Completada
F7 · Pruebas, deploy y cierre	S18–S20	25	Completada
Total		325	Cerrado

4.2— Metodología de trabajo

Se aplicó una metodología incremental con ciclos cortos por fase:

- definición de alcance y criterios de aceptación,
- implementación por módulos,
- validación automatizada (lint + tests),
- verificación funcional y documental,
- cierre de fase con artefactos de evidencia.

Este enfoque permitió absorber cambios sin comprometer la estabilidad del sistema.

4.3– Estimación de costes

La estimación económica se realizó mediante coste-hora por rol junior, alineada con referencias del informe de perfiles TIC de la Junta de Andalucía (2018).

Cuadro 4.2: Coste estimado del desarrollo

BLOQUE	HORAS	COSTE (EUR)
Análisis y diseño	45	1.490,40
Infraestructura	30	861,60
Backend + frontend	165	4.738,80
IA, optimizador y scraping	35	1.005,20
Portal business y app móvil	25	718,00
Pruebas y despliegue	25	594,25
Total	325	9.408,25

Adicionalmente, el coste operativo en staging se mantuvo bajo durante el TFG gracias al uso de planes gratuitos o de bajo consumo (Render, Sentry, GitHub Actions).

4.4– Riesgos y mitigaciones

Los riesgos operativos principales fueron:

- **Cambios en fuentes de scraping:** mitigado con spiders modulares y fallback de crowdsourcing.
- **Dependencia de APIs externas:** mitigado con caché y degradación controlada.
- **Carga de trabajo en fases finales:** mitigado con cierre incremental de entregables y automatización de validación.

4.5– Resultado de planificación

La planificación permitió completar el alcance funcional priorizado del TFG, incluyendo arquitectura, implementación, pruebas y despliegue de staging, manteniendo coherencia entre roadmap, requisitos y evidencias de verificación.

Análisis de requisitos

5.1— Actores del sistema

El sistema define cuatro actores principales:

- **Consumidor:** crea listas, compara precios, optimiza rutas, usa OCR y consulta al asistente.
- **Comercio/PYME:** gestiona precios y promociones desde el portal business.
- **Administrador:** verifica perfiles, modera datos y supervisa la operación.
- **Sistema automatizado:** tareas de scraping y mantenimiento en segundo plano.

5.2— Requisitos de información

El modelo de información se articula en doce bloques (RI-001 a RI-012), que cubren:

- usuarios y preferencias de optimización,
- catálogo (productos, categorías, marcas),
- tiendas geolocalizadas y cadenas,
- precios actuales e históricos,
- listas de compra y resultados de optimización,
- perfiles business y promociones,
- sesiones OCR y conversaciones del asistente.

5.3— Requisitos funcionales

Se definieron 35 requisitos funcionales (RF-001 a RF-035) agrupados por dominios:

Cuadro 5.1: Agrupación de requisitos funcionales

DOMINIO	IDS	CAPACIDAD PRINCIPAL
Autenticación y perfil	RF-001_RF-005	Registro, login JWT, preferencias
Catálogo de productos	RF-006_RF-010	Búsqueda, detalle, crowdsourcing
Tiendas y mapa	RF-011_RF-014	Proximidad, detalle, favoritos
Precios	RF-015_RF-019	Comparación, histórico, alertas
Listas de compra	RF-020_RF-023	CRUD, compartir, plantillas
Optimizador	RF-024_RF-027	Ruta multicriterio y recálculo
OCR	RF-028_RF-029	Captura y matching fuzzy
Asistente LLM	RF-030_RF-031	Consultas naturales y sugerencias
Portal business	RF-032_RF-034	Onboarding, precios, promociones
Notificaciones	RF-035	Push / email por eventos

5.4– Criterios de aceptación del producto

Los criterios de aceptación se validan a nivel funcional y de experiencia:

- flujo de alta y acceso operativo en menos de 3 minutos,
- comparación de cesta completa en entorno de tiendas cercanas,
- optimización con top-3 rutas y desglose por parada,
- OCR con confirmación/edición previa a la inserción en lista,
- portal PYME con gestión de precios y promociones,
- notificaciones de eventos críticos (alertas, promociones, cambios compartidos).

5.5– Requisitos no funcionales

Se definieron ocho requisitos no funcionales (RNF-001 a RNF-008):

- rendimiento API (p95 bajo para operaciones CRUD y optimización),
- disponibilidad de staging,
- seguridad (JWT, HTTPS, rate limiting),
- usabilidad (flujo principal corto y criterios de accesibilidad),
- escalabilidad arquitectónica,
- mantenibilidad con cobertura y convenciones,
- compatibilidad multiplataforma,
- cumplimiento RGPD.

5.6— Reglas de negocio

Las reglas RN-001..RN-010 gobiernan límites y políticas operativas:

- radio máximo de búsqueda y número máximo de paradas,
- caducidad y prioridad de fuentes de precio,
- restricciones del asistente al dominio de compra,
- verificación previa de perfiles business,
- políticas de scraping responsable y frecuencia de ejecución.

5.7— Diagramas de casos de uso

Los diagramas de casos de uso recogen las interacciones principales de cada actor con el sistema.

5.8— Trazabilidad

La trazabilidad entre requisitos, implementación y pruebas se mantiene en:

- docs/memoria/07-requisitos.md (especificación completa),
- TASKS.md (estado por fase/tarea),
- tests/unit, tests/integration y frontend/web/e2e (verificación ejecutable).

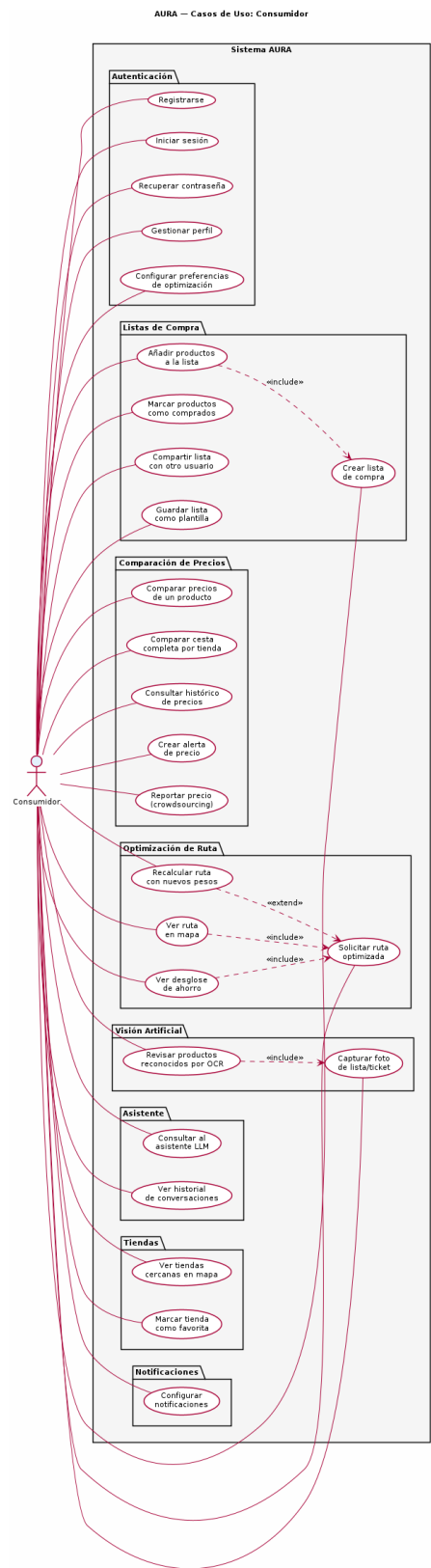


Figura 5.1: Casos de uso del Consumidor

AURA — Casos de Uso: Comercio / PYME y Administrador

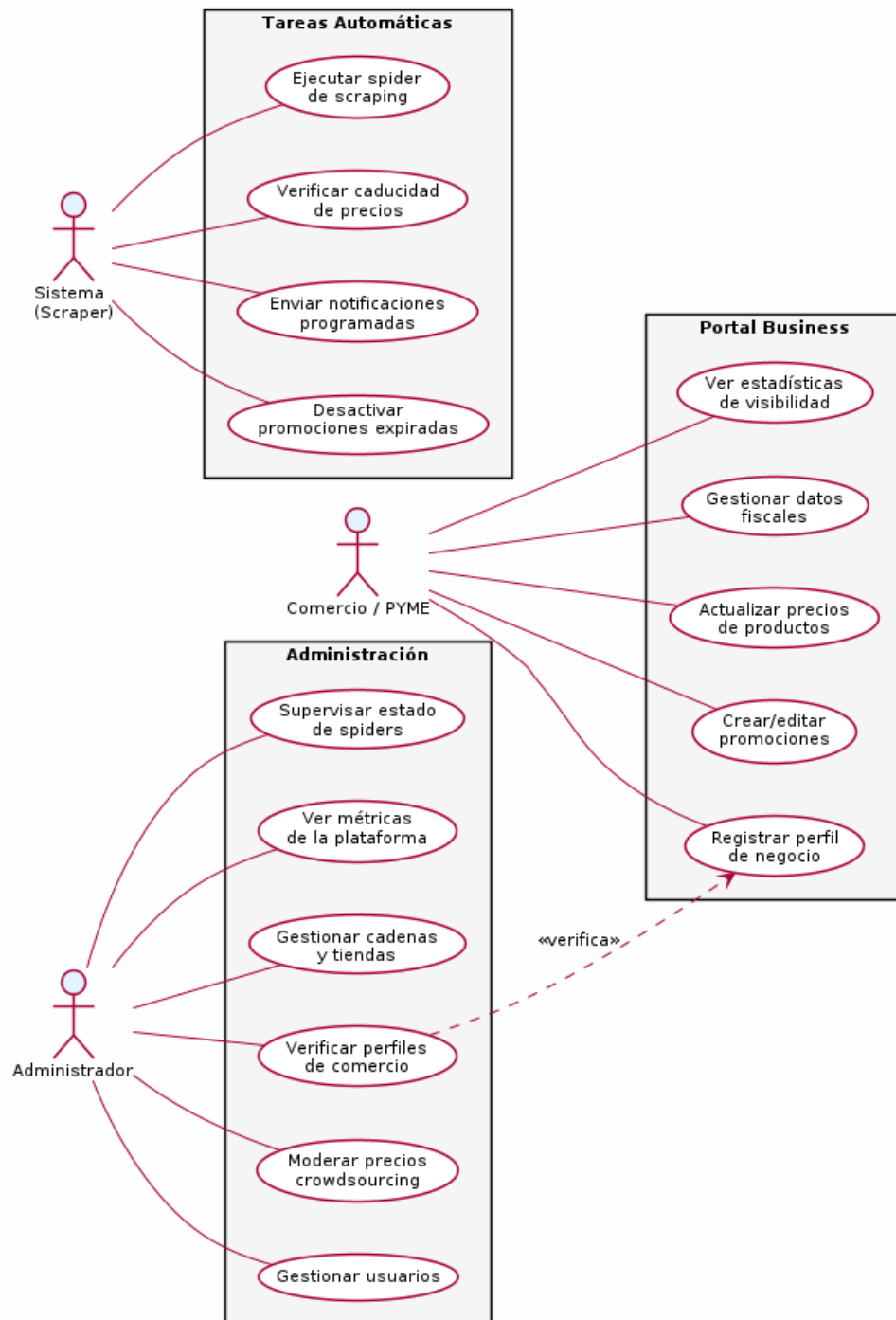


Figura 5.2: Casos de uso del Comercio y Administrador

Diseño e Implementación

6.1— Arquitectura del Sistema

6.1.1. Visión general

BargAIn sigue una arquitectura de **tres capas** clásica (presentación, lógica de negocio y datos), desplegada mediante contenedores Docker y organizada internamente según los principios de la **arquitectura hexagonal** (puertos y adaptadores). Esta elección permite desacoplar la lógica de dominio de los detalles de infraestructura — bases de datos, APIs externas, brokers de mensajería — facilitando el testing independiente de cada componente.

La arquitectura global se divide en cinco bloques principales:

1. **Aplicación cliente:** React Native (Expo) para iOS y Android, con una aplicación web React para el Portal Business de PYMEs.
2. **API Backend:** Django REST Framework como núcleo de la lógica de negocio, expuesto bajo HTTPS mediante Gunicorn + Nginx.
3. **Capa de datos:** PostgreSQL 16 con extensión PostGIS 3.4 para consultas geoespaciales.
4. **Capa de tareas asíncronas:** Celery con Redis como broker, para scraping periódico, envío de notificaciones, procesamiento OCR y cálculos del optimizador.
5. **Integraciones y servicios auxiliares:** Google Gemini API (asistente LLM), OpenRouteService (cálculo de distancias reales), Google Cloud Vision API (OCR) y Sentry (monitorización de errores).

6.1.2. Patrón de organización interna del backend

Cada aplicación Django del backend sigue una estructura modular consistente:

```
apps/<modulo>/
|-- models.py           # Entidades de dominio (ORM)
|-- serializers.py       # Transformacion de datos (entrada/salida)
|-- views.py            # ViewSets y vistas (controladores)
|-- urls.py             # Rutas del modulo
|-- services.py         # Logica de negocio
|-- tasks.py            # Tareas Celery asincronas
```

```
-- permissions.py      # Logica de autorizacion
-- tests/
```

La separación entre `views.py` y `services.py` es deliberada: los ViewSets se limitan a la gestión HTTP (autenticación, serialización, códigos de respuesta), mientras que la lógica de negocio reside exclusivamente en los servicios, lo que permite invocarla desde tareas Celery o tests sin simular peticiones HTTP.

La Figura 6.1 resume la distribución por capas de la solución.

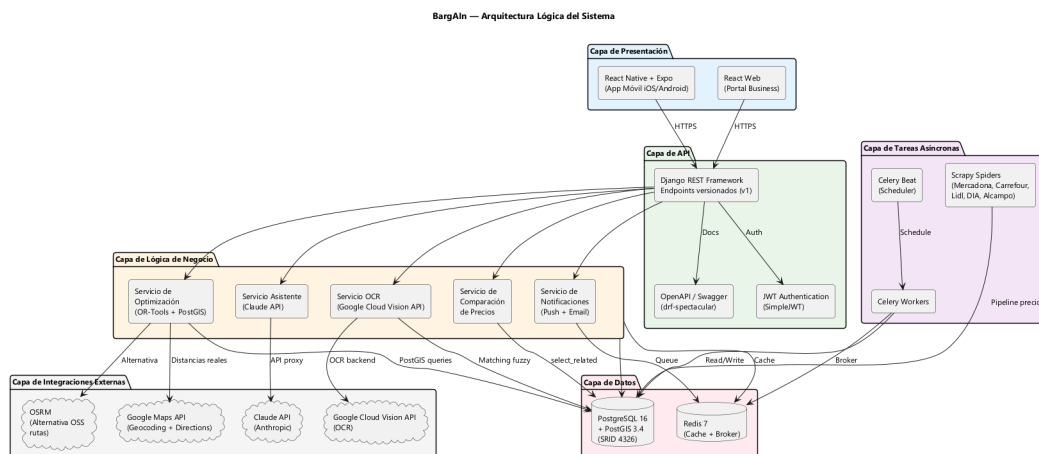


Figura 6.1: Arquitectura por capas de BargAI

6.1.3. Comunicación entre componentes

- **Frontend → Backend:** API REST con autenticación JWT. Axios gestiona la renovación automática del token de acceso ante un 401 Unauthorized.
- **Backend → Celery:** Encolar tareas mediante `.delay()` o `.apply_async()` sobre el broker Redis.
- **Backend → PostgreSQL:** Django ORM con consultas geoespaciales mediante `django.contrib.gis`.
- **Backend → APIs externas:** Clientes HTTP aislados en la capa de servicios de cada módulo (p.ej. `apps/optimizer/services/distance.py`) para Google Places, Gemini API y OpenRouteService.

6.2— Modelo de Datos

6.2.1. Módulo de usuarios

El modelo `User` extiende `AbstractBaseUser` de Django para permitir el login mediante email. Se definen tres roles: `consumer`, `business` y `admin`. El campo `default_location` utiliza `PointField(SRID=4326)`.

Las preferencias de optimización (`w_precio`, `w_distancia`, `w_tiempo`) se almacenan como decimales con restricción de que su suma sea igual a 1.0, validada a nivel de serializer:

```
def validate(self, data: dict) -> dict:
    weights = [
```

```

        data.get('weight_price', 0),
        data.get('weight_distance', 0),
        data.get('weight_time', 0),
    ]
    if abs(sum(weights) - 1.0) > 0.001:
        raise serializers.ValidationError(
            "Los pesos de optimizacion deben sumar exactamente 1.0"
        )
    return data

```

6.2.2. Módulo de productos

La búsqueda de productos por nombre utiliza **matching fuzzy** mediante la extensión PostgreSQL `pg_trgm` (trigramas). El campo `normalized_name` almacena el nombre normalizado (minúsculas, sin tildes, sin caracteres especiales) con índices GIN de trigramas:

```

def search_products(query: str, limit: int = 20) -> QuerySet[Product]:
    return (
        Product.objects
        .annotate(similarity=TrigramSimilarity(
            'normalized_name', normalize(query)))
        .filter(similarity__gt=0.3, is_active=True)
        .order_by('-similarity')[:limit]
    )

```

6.2.3. Módulo de tiendas

El campo `location` de `Store` es un `PointField` con índice GiST para búsquedas geoespaciales eficientes:

```

def get_stores_in_radius(
    lat: float, lng: float, radius_km: float
) -> QuerySet[Store]:
    user_location = Point(lng, lat, srid=4326)
    return (
        Store.objects
        .filter(
            location__dwithin=(user_location, D(km=radius_km)),
            is_active=True
        )
        .annotate(distance=Distance('location', user_location))
        .order_by('distance')
    )

```

El uso de `ST_DWithin` en lugar de `ST_Distance` en el `WHERE` es deliberado: `ST_DWithin` puede aprovechar el índice GiST, mientras que calcular distancias en el filtro forzaría un escaneo completo de la tabla.

6.2.4. Módulo de precios

El histórico de precios se materializa automáticamente: cada precio es inmutable una vez creado. La actualización de un precio supone la inserción de un nuevo registro. Una tarea Celery periódica (`expire_stale_prices`) marca como expirados los precios de scraping con más de 48 horas de antigüedad y los de crowdsourcing con más de 24 horas.

6.2.5. Módulo del optimizador

El módulo del optimizador incluye tres entidades relacionales para almacenar el resultado de la optimización de forma trazable:

- **OptimizationResult:** resultado global con pesos, modo y métricas totales. Incluye `route_data` JSONB para compatibilidad de lectura rápida.
- **OptimizationRouteStop:** cada parada de la ruta con tienda, orden, distancia, tiempo y subtotal.
- **OptimizationRouteStopItem:** ítem concreto por parada, con producto resuelto, precio elegido, texto original y puntuación de similitud.

6.2.6. Índices clave

Cuadro 6.1: Índices principales de la base de datos

TABLA	COLUMNA(S)	TIPO	JUSTIFICACIÓN
store	location	GIST	Búsquedas geoespaciales por radio
price	(product, store, created_at)	B-Tree	Histórico y precio actual
price	expires_at	B-Tree parcial	Filtrar precios vigentes
product	normalized_name	GIN (trigram)	Búsqueda fuzzy por nombre
shopping_list_item	(shopping_list, is_checked)	B-Tree	Ítems pendientes

6.2.7. Modelo entidad-relación

La Figura 6.2 muestra el diagrama entidad-relación completo de la base de datos.

6.2.8. Diagrama de clases

La Figura 6.3 detalla las clases principales del dominio y sus relaciones.

6.3– Diseño de la API REST

6.3.1. Estructura general

La API sigue las convenciones REST y está versionada bajo el prefijo `/api/v1/`. Todos los endpoints requieren autenticación JWT salvo los de registro y login. Las respuestas siguen el formato unificado:

```
// Respuesta exitosa
{
  "success": true,
  "data": { ... },
  "meta": { "count": 42, "page": 1, "page_size": 20 }
}

// Respuesta de error
{
  "success": false,
  "error": {
```

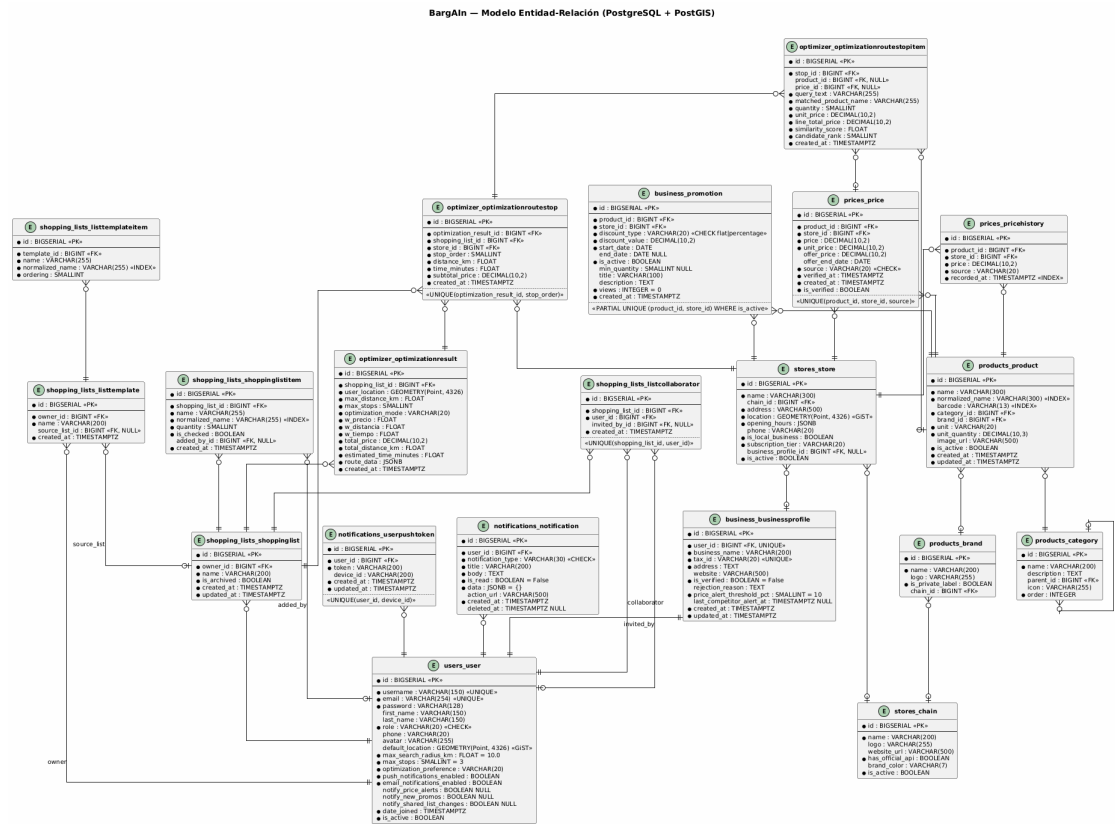



Figura 6.2: Diagrama entidad-relación de BargAIn

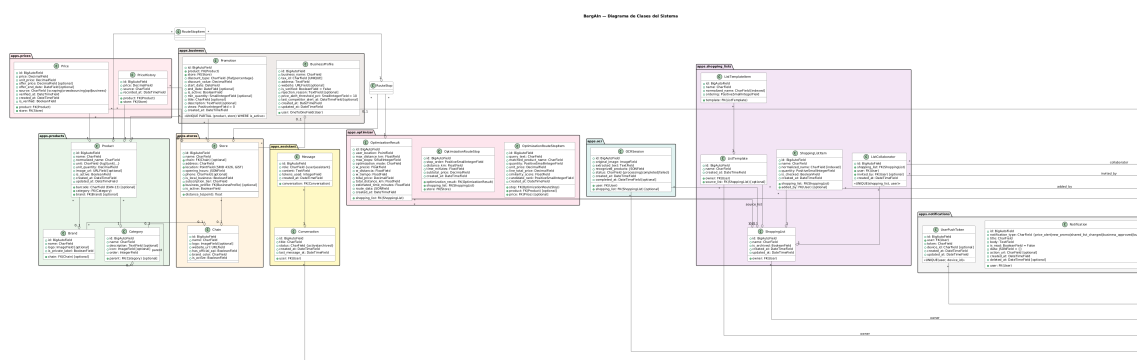


Figura 6.3: Diagrama de clases del dominio

```

    "code": "STORE_NOT_FOUND",
    "message": "No se encontraron tiendas en el radio especificado",
    "details": {}
  }
}

```

6.3.2. Endpoints principales

Cuadro 6.2: Endpoints de autenticación y usuarios

MÉTODO	ENDPOINT	DESCRIPCIÓN
POST	/api/v1/auth/register/	Registro de nuevo usuario
POST	/api/v1/auth/login/	Login: access + refresh token
POST	/api/v1/auth/token/refresh/	Renovar access token
POST	/api/v1/auth/logout/	Invaldar refresh token
GET/PATCH	/api/v1/users/me/	Perfil del usuario autenticado
GET/PATCH	/api/v1/users/me/preferences/	Preferencias de optimización

Cuadro 6.3: Endpoints del optimizador, OCR y asistente

MÉTODO	ENDPOINT	DESCRIPCIÓN
POST	/api/v1/optimize/	Calcular o recalcular ruta óptima
GET	/api/v1/optimize/?shopping_list_id={id}	Cargar última ruta persistida
POST	/api/v1/ocr/scan/	Procesar imagen y devolver ítems reconocidos
GET/POST	/api/v1/assistant/conversations/	Listar y crear conversaciones
POST	/api/v1/assistant/conversations/{id}/messages/	Enviar mensaje al asistente

6.3.3. Autenticación y autorización

Se implementa un sistema de permisos a tres niveles:

- **IsAuthenticated:** Aplicado por defecto a todos los endpoints privados.
- **IsBusinessOwner:** Permite operar solo sobre el perfil de negocio propio.
- **IsAdminUser:** Para endpoints de administración del sistema.
- **IsShoppingListOwnerOrShared:** Permite acceso al propietario y colaboradores invitados.

Los tokens JWT se generan con `django-rest-framework-simplejwt`. El token de acceso tiene validez de 60 minutos y el de refresco de 7 días con rotación automática.

6.4— Implementación del Backend

6.4.1. Módulo de notificaciones

El módulo gestiona el buzón de notificaciones del usuario (inbox) y el envío de notificaciones push mediante **Expo Push Notifications**. Las entregas se procesan asincrónicamente con tareas Celery.

- **Notification:** Registro persistente con `notification_type` (`price_alert`, `new_promo`, `shared_list_changed`, `business_approved`, `business_rejected`), campos `title/body`, bandera `is_read`, data JSONB para payload variable, `action_url` para deep links y `deleted_at` para soft-delete.
- **UserPushToken:** Token Expo registrado por dispositivo. La clave única (`user`, `device_id`) evita duplicados al reinstalar la app.

6.4.2. Módulo de portal business

- **BusinessProfile:** Perfil verificable con `tax_id` (CIF/NIF único), estado de verificación y umbral de alerta de competidor (`price_alert_threshold_pct`).
- **Promotion:** Promoción temporal de un producto en una tienda. Restricción parcial única (`product`, `store`) WHERE `is_active` garantiza que no existan dos promociones activas para el mismo par.

La tarea Celery `check_competitor_prices` se ejecuta cada 6 horas y alerta al propietario cuando detecta diferencias superiores al umbral configurado.

6.5— El Algoritmo de Optimización de Rutas

6.5.1. Formulación del problema

El algoritmo de optimización es la aportación técnica central del TFG. El problema es una variante del **Vehicle Routing Problem (VRP)** con objetivos múltiples, resuelto mediante una solución heurística eficiente.

Entrada:

- Lista de la compra $L = \{i_1, i_2, \dots, i_n\}$ (n ítems textuales)
- Ubicación del usuario $u = (lat, lng)$
- Radio máximo r (km), número máximo de paradas k (1–4)
- Pesos de preferencia w_{precio} , $w_{distancia}$, w_{tiempo} (suman 1.0)

Salida: Top-3 asignaciones de ítems resueltos a tiendas que minimizan la función de coste.

6.5.2. Función de scoring multicriterio

$$\text{Score}(A) = w_{precio} \cdot \text{ahorro_norm}(A) - w_{distancia} \cdot \text{dist_extra_norm}(A) - w_{tiempo} \cdot \text{tiempo_extra_norm}(A) \quad (6.1)$$

Donde $\text{ahorro_normalizado} = (\text{coste_single_best} - \text{coste}(A)) / \text{coste_single_best} \in [0, 1]$ y $\text{distancia_extra_normalizada} = \max(0, \text{dist_ruta}(A) - \text{dist_single}) / (r \times 1,5)$.

6.5.3. Pipeline de ejecución

El algoritmo se ejecuta en seis fases secuenciales dentro de `apps/optimizer/services.py`:

1. **Fase 0 — Resolución textual de ítems:** Preselección por trigramas sobre `Product.normalized_name` ranking fuzzy (`token_set_ratio`), y validación contextual por disponibilidad de precio.

- 2. **Fase 1 — Ingesta de precios candidatos:** Para cada producto resuelto, se obtienen los precios vigentes en tiendas dentro del radio con una sola consulta optimizada con `select_related` y filtros geoespaciales.
- 3. **Fase 2 — Pre-filtrado de tiendas:** Se limita a 30 tiendas candidatas máximo para controlar la explosión combinatoria ($C(30, 3) = 4.060$ combinaciones para $k=3$).
- 4. **Fase 3 — Generación y evaluación de asignaciones:** Para cada combinación de tiendas, asignación greedy óptima de ítems al menor precio disponible.
- 5. **Fase 4 — Cálculo de distancias reales:** Matriz de distancias vía **OpenRoute-Service (ORS)** en una sola petición. Fallback haversine si ORS no está disponible:

```
POST https://api.openrouteservice.org/v2/matrix/driving-car
Authorization: {ORS_API_KEY}

{
  "locations": [[lng1, lat1], [lng2, lat2], ...],
  "metrics": ["distance", "duration"],
  "units": "km"
}
```

- 6. **Fase 5 — Scoring y ranking:** Normalización de valores al rango $[0, 1]$ y devolución de las top-3 asignaciones ordenadas por score descendente.

6.5.4. Diagrama de secuencia del optimizador

La Figura 6.4 ilustra el flujo de mensajes entre componentes durante una petición de optimización.

6.5.5. Integración con OR-Tools

Para combinaciones complejas donde el problema de asignación supera la heurística greedy, se utiliza **OR-Tools de Google** para resolver el problema como programación lineal entera (ILP), garantizando la optimalidad de la asignación de productos a tiendas dentro de un subconjunto fijo de tiendas.

6.5.6. Complejidad y rendimiento

Cuadro 6.4: Tiempos estimados de ejecución del optimizador

ESCENARIO	TIENDAS	COMBINACIONES (K = 3)	TIEMPO
Lista pequeña (5 productos)	≤ 15	455	< 1 s
Lista media (20 productos)	≤ 30	4.060	< 3 s
Lista grande (50 productos)	≤ 30	4.060	< 5 s

El umbral de 5 s (RNF-001) se cumple gracias a: pre-filtrado de tiendas, caché de matriz de distancias en Redis, y ejecución como tarea Celery asíncrona.

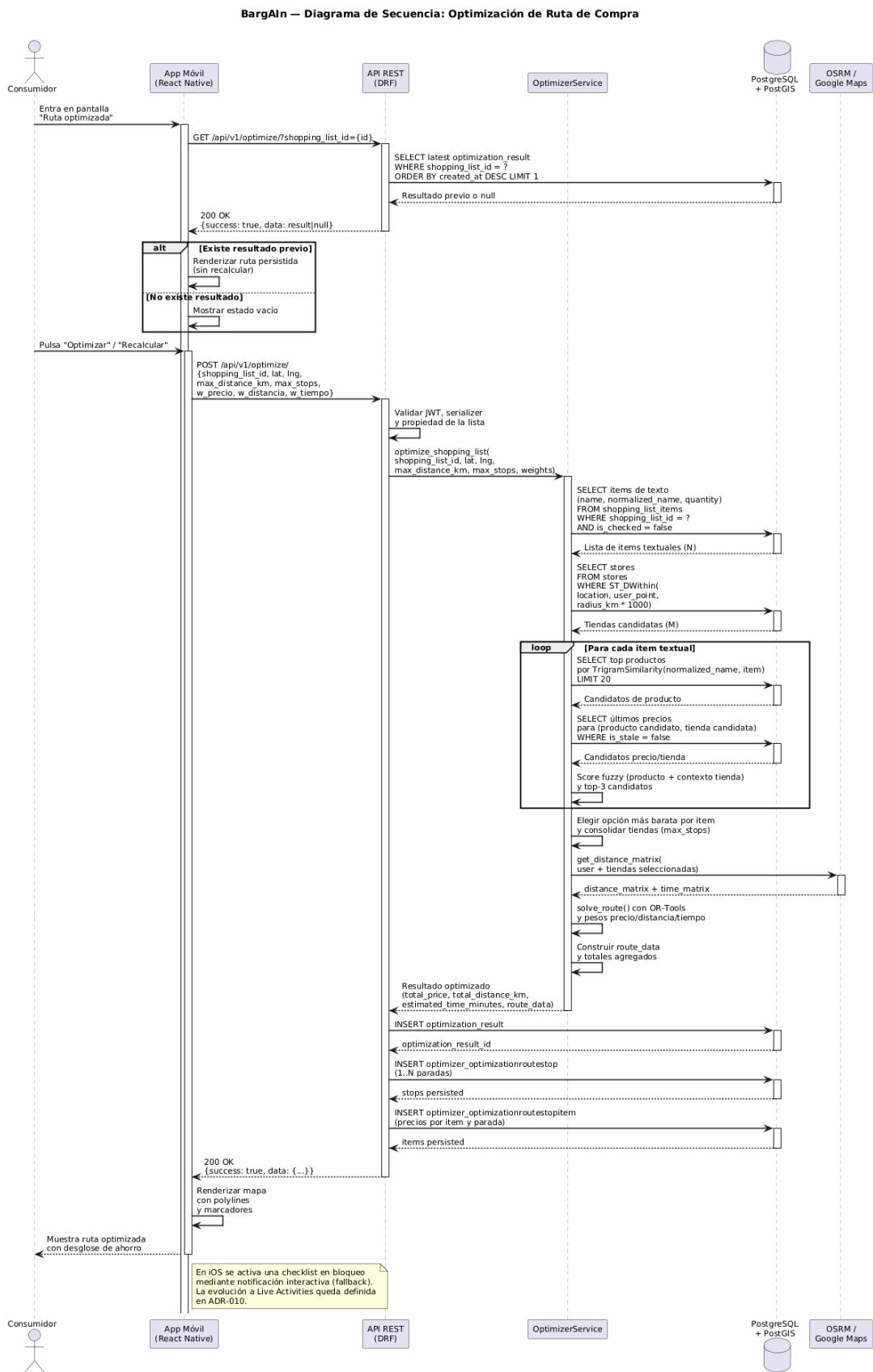


Figura 6.4: Diagrama de secuencia del proceso de optimización de ruta

6.6— Módulo de Web Scraping

El scraper es un **proyecto Scrapy independiente** (`scraping/`) desacoplado del backend Django, que comunica con la base de datos a través de la API REST del propio backend.

Cada cadena requiere una estrategia diferente:

- **Mercadona:** API interna (`tienda.mercadona.es/api/`) no oficial pero estable, accesible mediante peticiones HTTP estándar.
- **Carrefour / Lidl / DIA:** Catálogos con renderizado del lado del cliente (SPA), por lo que se usa **Playwright** para ejecutar el JavaScript y obtener el DOM hidratado.

Los spiders se programan con **Celery Beat**, con un delay mínimo de 2 segundos entre peticiones consecutivas (regla de negocio RN-010).

6.7— Módulo OCR

El backend invoca **Google Cloud Vision API** con la imagen capturada por el usuario. La decisión de migrar desde Tesseract a Vision API se adoptó tras comprobar que Tesseract no reconoce con suficiente claridad tickets y listas fotografiadas en condiciones reales:

```
from google.cloud import vision

def extract_text_from_image(image_bytes: bytes) -> list[str]:
    """Extrae líneas de texto mediante Google Cloud Vision API."""
    client = vision.ImageAnnotatorClient()
    image = vision.Image(content=image_bytes)
    response = client.text_detection(image=image)
    full_text = response.full_text_annotation.text
    return [line.strip() for line in full_text.splitlines() if line.strip()]
```

Cada línea extraída se normaliza y se busca en el catálogo mediante el servicio de matching fuzzy de `apps/products`. Se devuelve una lista de candidatos con nivel de confianza.

6.7.1. Diagrama de secuencia OCR

La Figura 6.5 muestra el flujo de procesamiento desde la captura de imagen hasta la inserción de ítems en la lista.

6.8— Integración del Asistente LLM

El asistente conversacional utiliza la **Google Gemini API** (`gemini-2.0-flash-lite`) como modelo subyacente (ADR-008). El backend actúa como proxy, añadiendo contexto de la compra del usuario al system prompt antes de enviar cada mensaje a la API mediante el SDK `google-genai`.

Se aplica una **ventana deslizante** sobre el historial de mensajes (últimos 20 mensajes + system prompt) para mantener el consumo de tokens controlado. El system prompt incluye un guardrail que limita las respuestas a consultas de compra doméstica (RN-007).

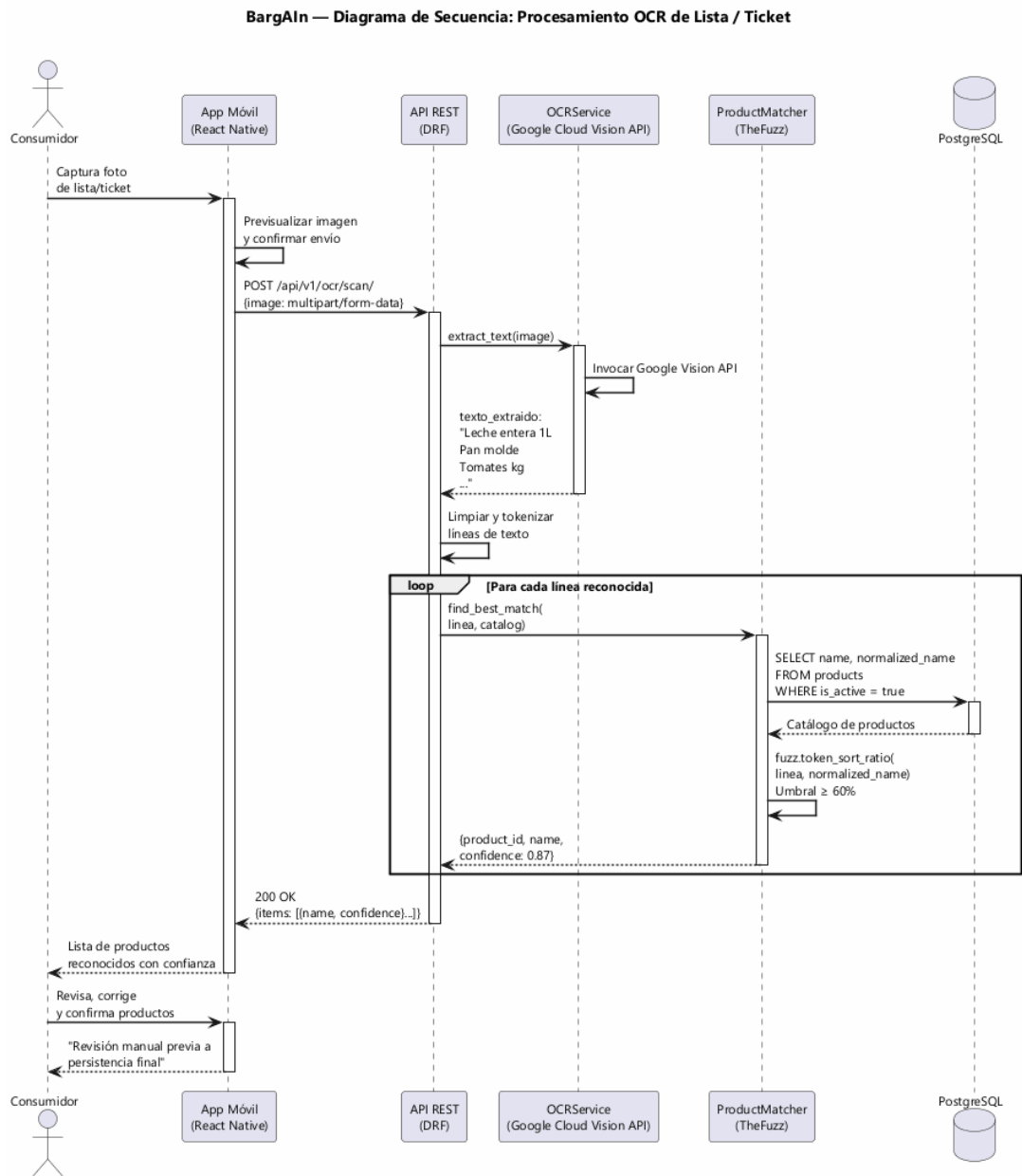


Figura 6.5: Diagrama de secuencia del flujo OCR

6.9— Infraestructura y Despliegue

6.9.1. Contenedores Docker

Cada componente se ejecuta en un contenedor independiente, orquestados con Docker Compose:

- **nginx:** Reverse proxy y servicio de estáticos.
- **backend:** Django + Gunicorn (3 workers).
- **celery-worker:** Workers para tareas asíncronas.
- **celery-beat:** Scheduler de tareas periódicas.
- **db:** PostgreSQL 16 + PostGIS.
- **redis:** Broker Celery + caché.

Para el entorno de desarrollo se usa el modelo híbrido (ADR-002): backend en Docker, frontend nativo en el host. Docker volumes en Windows rompen el HMR de Metro/Expo, por lo que el frontend no se dockeriza.

6.9.2. Pipeline CI/CD

El repositorio incluye tres workflows de GitHub Actions:

- **ci-backend.yml:** Ejecuta `ruff check`, `ruff format --check` y `pytest --cov` en cada push a cualquier rama.
- **ci-frontend.yml:** Ejecuta `eslint`, `prettier --check` y `jest --coverage` en cada push.
- **ios-build.yml:** Genera el `.ipa` unsigned mediante `expo prebuild + xcodebuild` en un runner `macos-latest`. El artefacto se descarga y se instala en el iPhone de demo con Sideloadly.

El deploy a staging se realiza mediante **Render Blueprints**. El repositorio define dos variantes declarativas: `render.yaml`, con web service, worker Celery, scheduler beat, Redis y base de datos PostgreSQL independientes; y `render.free.yaml`, la variante free tier empleada durante el TFG, que consolida Gunicorn y un proceso Celery worker + beat embebido en un único web service. Cada push a `main` dispara un redespiegue automático.

6.9.3. Variables de entorno

La configuración sensible (claves API, credenciales de BD, secret key de Django) se gestiona exclusivamente mediante variables de entorno, nunca hardcodeadas. Se proporciona `.env.example` con los nombres de todas las variables requeridas.

6.10— Diseño de la Interfaz de Usuario

6.10.1. Sistema de diseño

El frontend define un sistema de diseño propio en `src/theme/` con los siguientes tokens:

Tipografía: Inter (sans-serif) con escala modular. **Espaciado:** Escala de 4px.

Cuadro 6.5: Paleta de colores del sistema de diseño

TOKEN	VALOR	USO
primary	 #2E7D32	Acciones principales, CTAs
primaryLight	 #60AD5E	Estados hover, fondos sutiles
secondary	 #FF6F00	Ahorros, destacados, ofertas
surface	 #FAFAFA	Fondo de tarjetas
error	 #C62828	Errores y alertas
textPrimary	 #212121	Texto principal
textSecondary	 #757575	Texto de apoyo

6.10.2. Pantallas principales

La aplicación se estructura en cinco pestañas principales:

1. **Inicio / Dashboard:** Resumen de la lista activa, precio estimado en la tienda más barata cercana, acceso rápido al asistente y alertas activas.
2. **Mi Lista:** Gestión de la lista activa con buscador de autocompletado, grupos por categoría, swipe-to-delete/check y botón de optimizar ruta.
3. **Explorar:** Mapa interactivo con marcadores de tiendas y panel inferior deslizable con detalles de la tienda seleccionada.
4. **Optimizador:** Sliders de pesos, cards comparativas de las top-3 rutas y visualización de la ruta en mapa con polylines y marcadores numerados.
5. **Perfil:** Datos personales, preferencias de optimización, configuración de notificaciones y acceso al historial del asistente.

6.10.3. Adaptación a uso web (responsive y conveniencias de escritorio)

La misma base de código Expo (`frontend/src`) se reutiliza para ejecución en navegador de escritorio mediante `react-native-web`, sin duplicar pantallas ni alterar la navegación. La adaptación a escritorio se implementa como una capa transversal sobre las 15 pantallas existentes, organizada en cuatro ejes:

1. **Diseño responsive.** El hook `useBreakpoint()` (`src/hooks/`) clasifica el ancho del viewport en `mobile` (<768 px), `tablet` ($768-1023$ px) y `desktop` (≥ 1024 px). Las pantallas centran su contenido a un ancho máximo legible (entre 560 y 900 px según la pantalla) y los listados de tarjetas pasan de 1 columna en móvil a 2-4 columnas en escritorio. El componente `MasterDetailLayout` aporta una disposición maestro-detalle de dos paneles, y la barra de pestañas inferior se centra a 900 px en escritorio.
2. **Interacción con ratón y teclado.** Se añaden estados `hover` (tintes y realces), atajos de teclado (`Cmd/Ctrl+K` para enfocar el buscador, `Enter` para añadir ítem, `Esc` para cerrar), `auto-focus` de campos al montar y `tooltips` web (`WebTooltip`). El reordenado de ítems de una lista emplea `drag-and-drop` HTML5 nativo en la variante `ListDetailScreen.web.tsx`.
3. **Conveniencias de escritorio.** Las utilidades de `src/utils/webExport.ts` permiten exportar la lista de la compra a `.txt`, la comparativa de precios a `.csv` y la ruta a fichero, así como copiar al portapapeles direcciones y enlaces. La generación de CSV incluye un `guard` de inyección de fórmulas (OWASP) que escapa las celdas que comienzan por `=`, `+`, `-` o `@`.

4. **Deep-linking.** La configuración `src/navigation/linking.ts` registra URLs estables para las pantallas (p.ej. `/app/lists/:listId`, `/app/map/store/:storeId`, `/app/home/catalog?q=...`), de modo que el estado de la aplicación es enlazable y navegable desde la barra de direcciones del navegador.

Todas las adiciones están condicionadas a la plataforma web (comprobación de `Platform.OS`) o al *breakpoint* activo, de forma que el comportamiento en móvil permanece inalterado. Estas mejoras se incorporaron en una fase específica de adaptación web posterior al cierre del plan inicial, y se validaron con 111 pruebas Jest (16 *suites*) y una batería de verificación manual en navegador.

Manual de Usuario

7.1— Introducción

Este capítulo describe, desde la perspectiva del usuario final, el funcionamiento completo de BargAIn en sus tres contextos de uso: la aplicación móvil para consumidores (iOS y Android), su versión de escritorio ejecutada en navegador, y el portal web orientado a pequeños y medianos comercios (PYMEs). La exposición sigue el recorrido natural de un usuario: desde el registro y el acceso, pasando por la gestión de listas de la compra y la comparación de precios, hasta las funcionalidades diferenciales del sistema —la optimización multicriterio de rutas de compra, el reconocimiento óptico de tickets y el asistente conversacional—, para concluir con las herramientas de gestión que ofrece el portal *business*.

Todas las imágenes que ilustran este capítulo son capturas reales de la aplicación desplegada en el entorno de desarrollo descrito en el Capítulo 6, obtenidas sobre un conjunto de datos de demostración representativo (catálogo normalizado, precios de varias cadenas de supermercados en Sevilla y un comercio local verificado). Las capturas de la aplicación móvil corresponden a un *viewport* de 390×844 puntos, equivalente al de un teléfono actual de gama media; las de escritorio, a una ventana de 1440×900.

7.2— Requisitos de uso

Para utilizar BargAIn como consumidor se requiere un dispositivo móvil iOS (14 o superior) o Android (10 o superior) con la aplicación instalada, o bien un navegador de escritorio moderno; una conexión a Internet activa; una cuenta de usuario registrada; y, para las funciones de mapa y optimización de rutas, el permiso de ubicación concedido. El portal para comercios es una aplicación exclusivamente web y solo precisa navegador.

En el entorno de desarrollo local, el sistema se levanta con el backend en Docker (`make dev`), la base de datos migrada (`make migrate-docker`) y el frontend Expo ejecutado de forma nativa en el host (`make frontend`), conforme al modelo híbrido adoptado en el ADR-002.

7.3— Registro y acceso al sistema

El acceso a la aplicación se articula en torno a un esquema clásico de autenticación con correo electrónico y contraseña sobre tokens JWT. La pantalla de inicio de sesión (Figura 7.1, izquierda) presenta un formulario deliberadamente austero —dos campos y una única acción principal— con el fin de minimizar la fricción de entrada. El formulario

de registro (Figura 7.1, derecha) solicita únicamente los datos imprescindibles para crear la cuenta: nombre, apellidos, correo y contraseña con confirmación, aplicando validación inmediata en el cliente antes de delegar en la validación definitiva del servidor.

The image displays two mobile application screens side-by-side. The left screen is the login interface, featuring the 'BARGAIN' logo at the top, the tagline 'Tu compra inteligente', and input fields for 'Usuario' (containing 'demo@bargain.local') and 'Contraseña' (masked with dots). Below these is an orange 'Iniciar sesión' button, a link for '¿Olvidaste tu contraseña?', and a link for '¿No tienes cuenta? Regístrate'. The right screen is the registration interface, also with the 'BARGAIN' logo and the heading 'Crea tu cuenta' with the subtext 'Únete a BargAln y empieza a ahorrar'. It contains input fields for 'Nombre de usuario' (containing 'Nico'), 'Nombre' (containing 'Parrilla'), 'Apellidos' (containing 'nico@example.'), and 'Email' (containing 'Demo1234!'). There are also fields for 'Contraseña' and 'Repetir' (both masked), with the 'Repetir' field containing the text 'Repetir'. An orange 'Crear cuenta' button is at the bottom, along with a link for '¿Ya tienes cuenta? Inicia sesión'.

Figura 7.1: Acceso al sistema: inicio de sesión (izquierda) y registro de un nuevo usuario (derecha)

Tras una autenticación satisfactoria, el sistema emite un par de tokens (acceso y refresco) y carga la navegación principal por pestañas. La sesión se mantiene de forma transparente: si el token de acceso expira, el interceptor HTTP lo renueva automáticamente con el token de refresco, de modo que el usuario solo vuelve a introducir credenciales si permanece inactivo más allá de la vida útil de este último. En el cliente móvil ambos tokens se custodian en el almacenamiento seguro del sistema operativo (*Keychain* en iOS, *Keystore* en Android).

7.4— Pantalla principal

La pantalla de inicio (Figura 7.2) actúa como cuadro de mando del usuario y condensa, en un único recorrido vertical, la información de mayor valor inmediato: un buscador global con acceso directo al catálogo; una botonera de acciones rápidas (plantillas, productos, escaneo OCR y favoritos); las notificaciones más recientes; las listas activas con su número de productos; las tiendas detectadas en el radio configurado; y las alertas de precio vigentes con su precio objetivo. Esta disposición responde a un principio de diseño que ha guiado toda la interfaz: las tareas frecuentes deben resolverse en un máximo de tres interacciones desde el arranque de la aplicación.

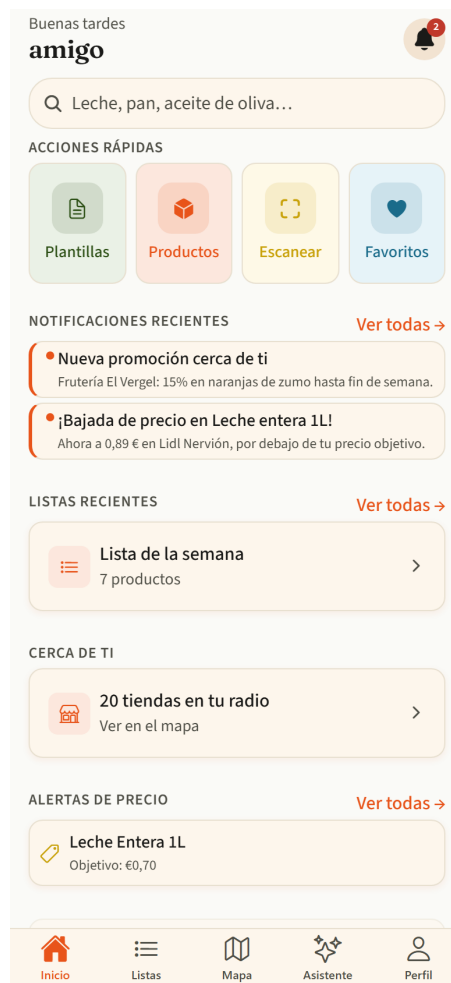


Figura 7.2: Pantalla principal de la aplicación móvil, con notificaciones, listas activas, tiendas cercanas y alertas de precio

7.5— Gestión de listas de la compra

La lista de la compra es la entidad vertebradora de BargAIn: sobre ella se calculan totales por tienda, se lanza la optimización de rutas y se articula la colaboración entre usuarios. La pestaña *Listas* (Figura 7.3, izquierda) muestra las listas del usuario con acciones directas de duplicado, conversión en plantilla y archivado. El detalle de una lista (Figura 7.3, derecha) presenta los ítems con controles de cantidad “+/-”, casillas de verificación para marcar productos como comprados y acciones de exportación y compartición en la cabecera.

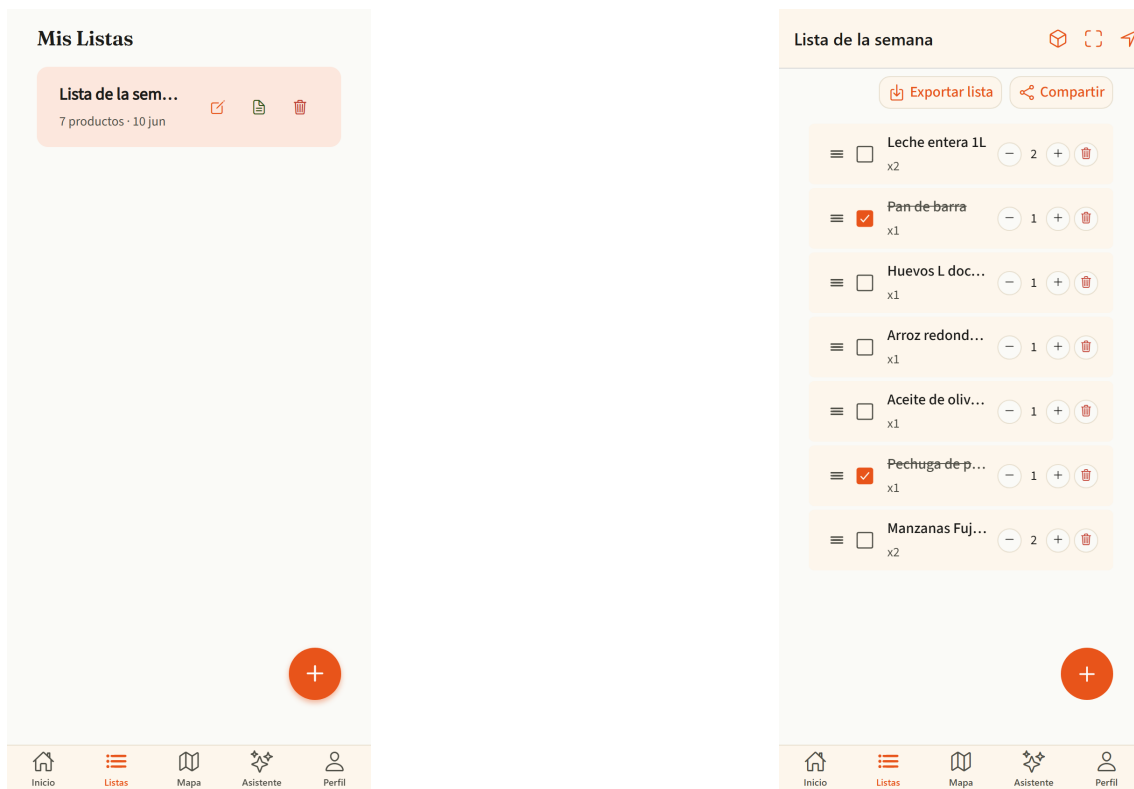


Figura 7.3: Gestión de listas: colección de listas del usuario (izquierda) y detalle de una lista con controles de cantidad y verificación (derecha)

El flujo de trabajo habitual es el siguiente: el usuario crea una lista con nombre personalizado; añade productos desde el catálogo, desde el buscador con autocompletado o mediante el escáner OCR; ajusta cantidades; y, durante la compra, marca cada producto adquirido. Una lista puede compartirse con hasta diez colaboradores, cuyas modificaciones se notifican al resto de participantes, y puede convertirse en plantilla para institucionalizar compras recurrentes. Las plantillas (Figura 7.4) funcionan como generadores de listas: a partir de una plantilla —por ejemplo, la compra básica semanal— se instancian nuevas listas con un solo gesto, evitando reconstruir manualmente colecciones de productos que apenas varían de una semana a otra.

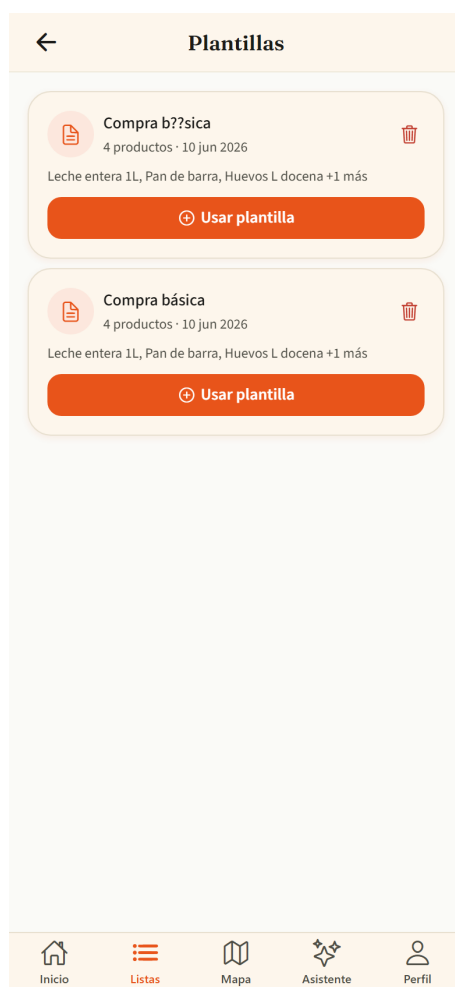


Figura 7.4: Plantillas de lista: instanciación de compras recurrentes a partir de una plantilla guardada

7.6— Catálogo y comparativa de precios

El catálogo (Figura 7.5, izquierda) ofrece una vista paginada del repositorio de productos normalizados, con filtros plegables por categoría, precio y distancia. Cada tarjeta sintetiza la información decisiva para el usuario: el precio mínimo detectado y la tienda que lo ofrece, junto con una acción de añadido rápido a la lista activa que se resuelve con un único toque.

Desde cualquier tarjeta se accede a la comparativa de precios del producto (Figura 7.5, derecha), que constituye una de las aportaciones centrales del sistema: una tabla ordenable con el precio del artículo en cada tienda del radio configurado, señalando las ofertas vigentes y la antigüedad de cada dato según su fuente (API oficial, *scraping* o aportación de usuarios, por este orden de prioridad). La misma pantalla incorpora el histórico de precios con un gráfico de evolución temporal, que permite valorar si el precio actual es coyuntural o estable, y la creación de alertas de precio sobre el producto consultado.

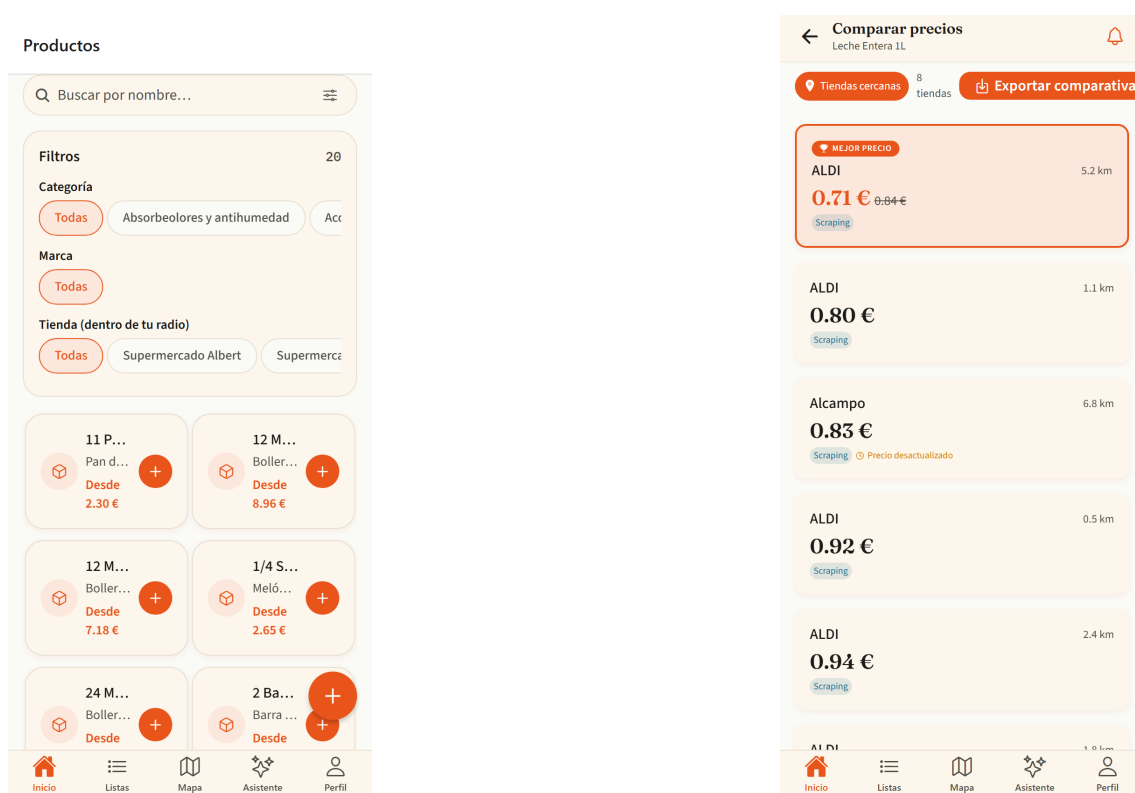


Figura 7.5: Catálogo de productos con filtros y añadido rápido (izquierda) y comparativa de precios por tienda de un producto (derecha)

El catálogo es deliberadamente abierto a la participación: cuando un usuario no encuentra un producto, puede proponer su alta mediante el formulario de la Figura 7.6, indicando nombre, marca, código de barras y categoría. La propuesta queda pendiente de revisión por un administrador, lo que preserva la calidad del repositorio común sin renunciar al crecimiento colaborativo del mismo.

Proponer producto

INFORMACIÓN DEL PRODUCTO

Nombre *

Ej. Leche semidesnatada 1L

Marca

Ej. Hacendado

Código EAN-13

Ej. 8412345678901

PRECIO

Precio (€)

0.00

Precio unitario (€/kg o €/L)

Ej. 1.20

Tienda

Supermercado Albert Supermercados El Jamón

Notas adicionales

Ej. Encontré este producto en el supermercado X pero no aparece en la app

Inicio Listas Mapa Asistente Perfil

Figura 7.6: Propuesta de alta de un nuevo producto en el catálogo, sujeta a aprobación por el administrador

7.7– Mapa de tiendas

La pestaña *Mapa* (Figura 7.7, izquierda) sitúa al usuario en el centro de su entorno comercial: sobre cartografía interactiva se representan las tiendas del radio configurado con marcadores diferenciados por cadena, y un panel inferior plegable lista los establecimientos visibles con su distancia y tiempo estimado de desplazamiento. Al seleccionar un establecimiento se abre su perfil (Figura 7.7, derecha), que reúne la información de contacto y horario junto con el catálogo de productos detectados en esa tienda y sus precios, de modo que el usuario puede evaluar un comercio concreto sin abandonar el contexto del mapa.

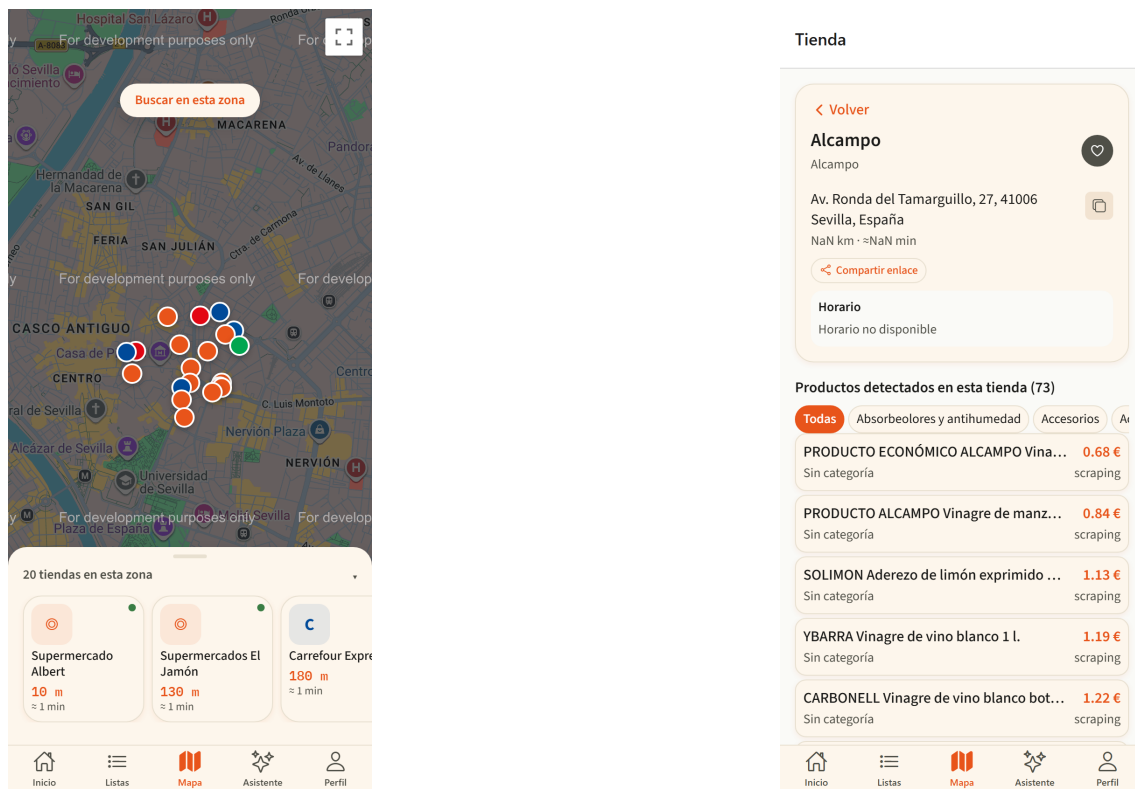


Figura 7.7: Mapa de tiendas con marcadores por cadena y panel de establecimientos (izquierda) y perfil de tienda con sus productos y precios (derecha)

El sistema permite marcar tiendas como favoritas, que se agrupan en una vista propia (Figura 7.8) accesible tanto desde el mapa como desde las acciones rápidas de la pantalla principal. Las tiendas favoritas reciben un tratamiento preferente en la generación de candidatos del optimizador de rutas, reflejando la fidelidad del usuario a determinados establecimientos.

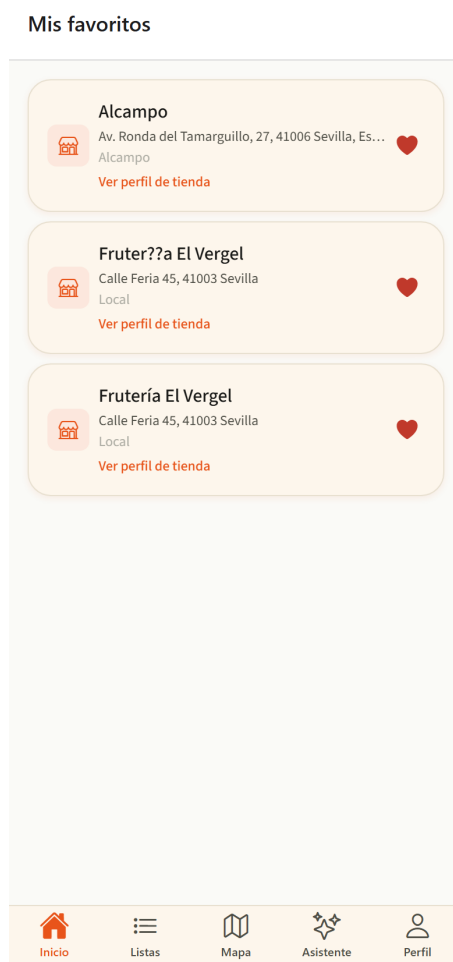


Figura 7.8: Tiendas favoritas del usuario, con acceso directo a su perfil

7.8— Alertas de precio y notificaciones

BargAIn invierte la carga de la vigilancia de precios: en lugar de obligar al usuario a consultar periódicamente la comparativa, es el sistema quien le avisa cuando se cumple una condición de interés. La bandeja de notificaciones (Figura 7.9, izquierda) centraliza tres familias de avisos —bajadas de precio que alcanzan el objetivo fijado, nuevas promociones de comercios cercanos y cambios en listas compartidas— con marcado de leídas individual o masivo, borrado lógico y enlaces profundos que conducen directamente a la pantalla afectada. Estos mismos avisos se entregan como notificaciones *push* en el dispositivo.

La gestión de alertas (Figura 7.9, derecha) muestra las alertas activas con su precio objetivo y permite editarlas o eliminarlas. Una alerta se crea desde la comparativa de un producto indicando el umbral deseado; cuando algún establecimiento publica un precio igual o inferior, el backend dispara la notificación correspondiente.

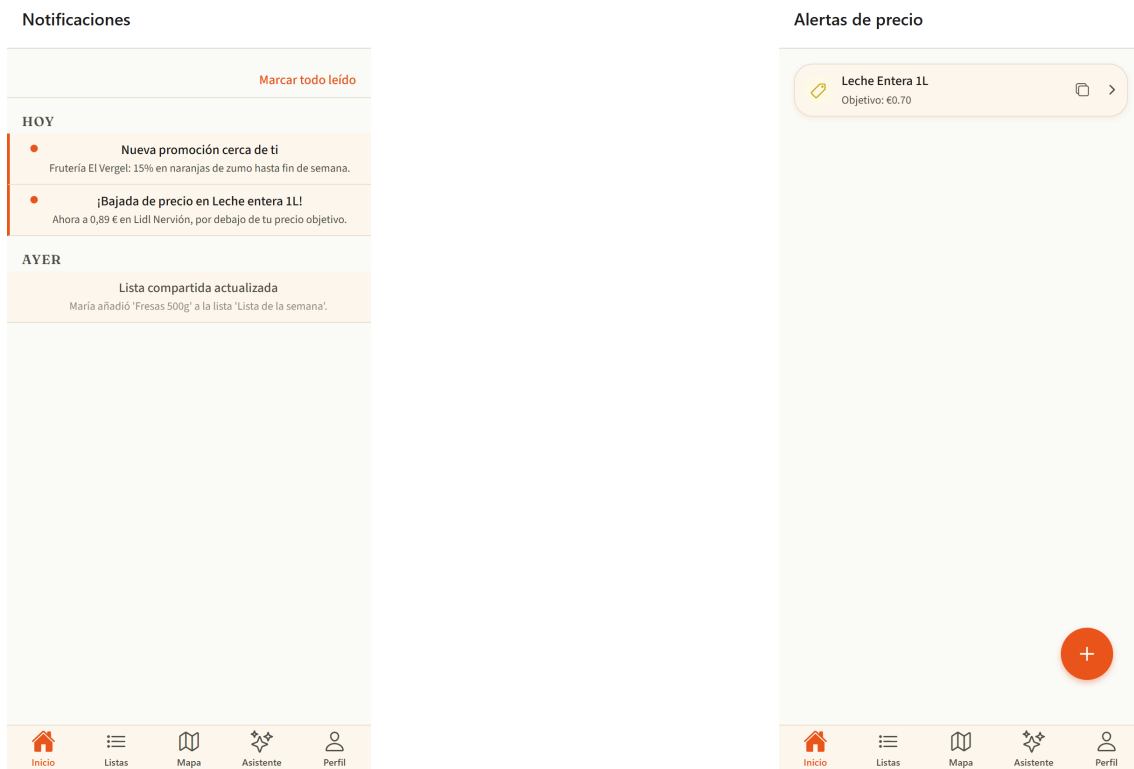


Figura 7.9: Bandeja de notificaciones con avisos de precio, promociones y listas compartidas (izquierda) y gestión de alertas de precio con su objetivo (derecha)

7.9— Perfil y preferencias del usuario

La pestaña *Perfil* (Figura 7.10, izquierda) agrupa la identidad del usuario y la configuración global de la aplicación: datos personales, preferencias de notificación y cierre de sesión, que invalida el token de refresco en el servidor. Su apartado más relevante para el comportamiento del sistema es la configuración del optimizador (Figura 7.10, derecha), donde el usuario gobierna la función de *scoring* multicriterio descrita en el Capítulo 6: el radio máximo de búsqueda (1–20 km), el número máximo de paradas por ruta y, sobre todo, los pesos relativos w_{precio} , $w_{\text{distancia}}$ y w_{tiempo} , ajustables mediante deslizadores cuya suma se normaliza automáticamente a la unidad. De este modo, una misma lista produce rutas distintas para un usuario que prioriza el ahorro frente a otro que prioriza la inmediatez, sin que ninguno de los dos necesite comprender el algoritmo subyacente.

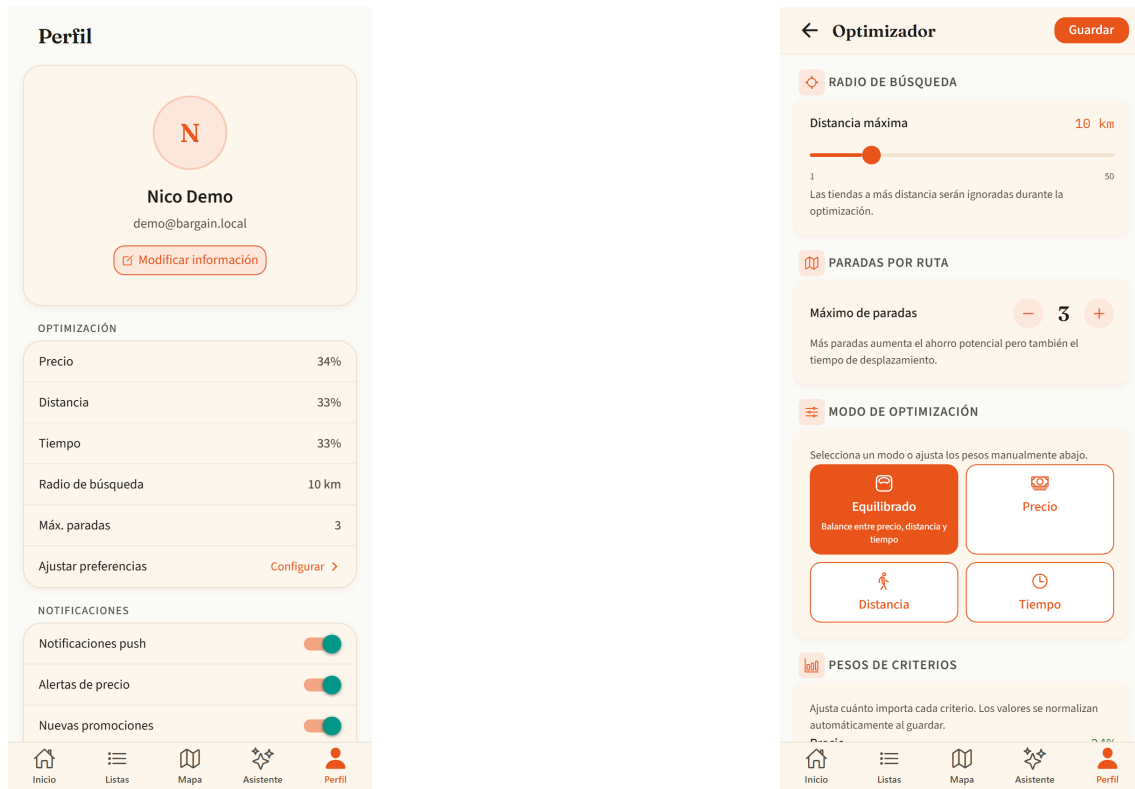


Figura 7.10: Perfil del usuario (izquierda) y configuración del optimizador con radio, paradas y pesos del *scoring* multicriterio (derecha)

Las operaciones sobre la cuenta se completan con la edición de los datos básicos y el cambio de contraseña (Figura 7.11), este último con verificación de la contraseña vigente y validación de robustez de la nueva.

Modificar información

Modificar información

Actualiza tus datos personales para mantener tu perfil al día.



Toca para cambiar la foto

Nombre

Apellidos

Email

Guardar cambios

Cambiar contraseña

Cambiar contraseña

Introduce tu contraseña actual y elige una nueva de al menos 8 caracteres.

Contraseña actual

Nueva contraseña

Confirmar contraseña

Guardar contraseña

Inicio Listas Mapa Asistente Perfil

Inicio Listas Mapa Asistente Perfil

Figura 7.11: Operaciones sobre la cuenta: edición del perfil (izquierda) y cambio de contraseña (derecha)

7.10— Ruta de compra optimizada

La optimización de rutas es la funcionalidad nuclear del proyecto y el punto donde convergen precios, geolocalización y preferencias del usuario. Desde el detalle de una lista, la acción *Ruta optimizada* conduce a la pantalla de la Figura 7.12, que resume el resultado de la última optimización persistida y permite recalcularla con la ubicación y los pesos actuales.

La cabecera recoge los parámetros empleados (radio y pesos), seguida del selector de paradas máximas (2–5) y del botón de recálculo. El bloque central presenta el resultado agregado —precio total estimado, distancia del recorrido, duración aproximada y número de paradas— y, a continuación, el desglose por parada: cada establecimiento de la ruta, con su distancia y tiempo parciales, enumera los productos que conviene adquirir en él junto con su precio. En el ejemplo de la figura, el optimizador ha asignado las manzanas a un comercio local porque su precio compensa el desvío conforme a los pesos configurados.

Cuando un ítem de la lista admite varias interpretaciones en el catálogo, la capa semántica respaldada por el modelo Gemini señala la ambigüedad y ofrece las alternativas (“Manzanas Fuji”, “Manzanas Golden”...), cada una con la acción *usar y recalcular*; la elección se persiste como preferencia de esa lista, de modo que las optimizaciones futuras ya no repiten la pregunta. El comportamiento de la pantalla es persistente: al salir y volver, se recupera la última ruta calculada desde la base de datos, y *Ver en mapa* abre el recorrido completo en la aplicación de mapas del dispositivo.



Figura 7.12: Ruta de compra optimizada: parámetros, resultado agregado, desglose por parada y resolución de ambigüedades semánticas con recálculo

7.11– Reconocimiento de tickets y listas (OCR)

El módulo OCR reduce drásticamente el coste de entrada de datos: en lugar de teclear una lista, el usuario fotografía un ticket de compra o una lista manuscrita y el sistema la convierte en ítems estructurados. El flujo sigue un asistente de tres pasos. En el primero (Figura 7.13, izquierda), la pantalla de captura ofrece la cámara o la galería como origen de la imagen. El segundo paso (Figura 7.13, derecha) presenta el resultado del reconocimiento —delegado en Google Cloud Vision y refinado con un emparejamiento difuso contra el catálogo—: cada línea detectada aparece con el texto original, el producto del catálogo con el que se ha emparejado y una etiqueta de confianza codificada por color (verde a partir del 80 %, amarillo entre el 50 y el 79 %, rojo por debajo). El usuario actúa como verificador final: corrige nombres en línea, ajusta cantidades con los selectores y descarta las líneas irrelevantes. En el tercer paso confirma la operación, eligiendo entre volcar los ítems a una lista existente o crear una nueva.

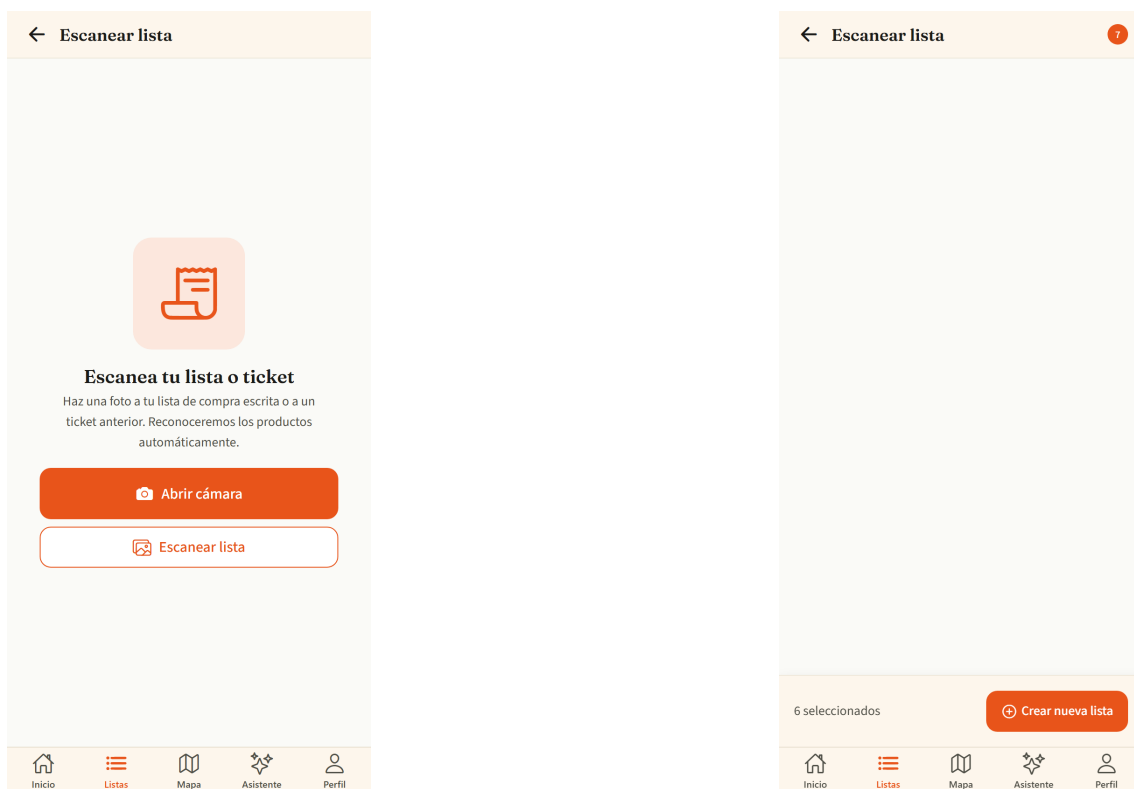


Figura 7.13: Asistente OCR: captura del ticket o lista (izquierda) y revisión de los ítems reconocidos con su nivel de confianza (derecha)

7.12– Asistente conversacional

El asistente (Figura 7.14) incorpora un modelo de lenguaje (Gemini, mediado por el backend conforme al ADR-008) especializado en la compra doméstica. La pantalla sigue el patrón conversacional estándar —mensajes del usuario a la derecha, respuestas del asistente a la izquierda— e incluye sugerencias de arranque que comunican el alcance del sistema: comparar precios entre cadenas, localizar la tienda más barata para un producto o pedir la optimización de la lista activa. Cuando una respuesta menciona productos, precios o tiendas, incorpora fichas interactivas que permiten añadir el producto a la lista sin abandonar la conversación. El asistente está acotado deliberadamente a su dominio:

ante preguntas ajenas a la compra responde declinando la consulta, como parte de las salvaguardas de uso descritas en el Capítulo 6.

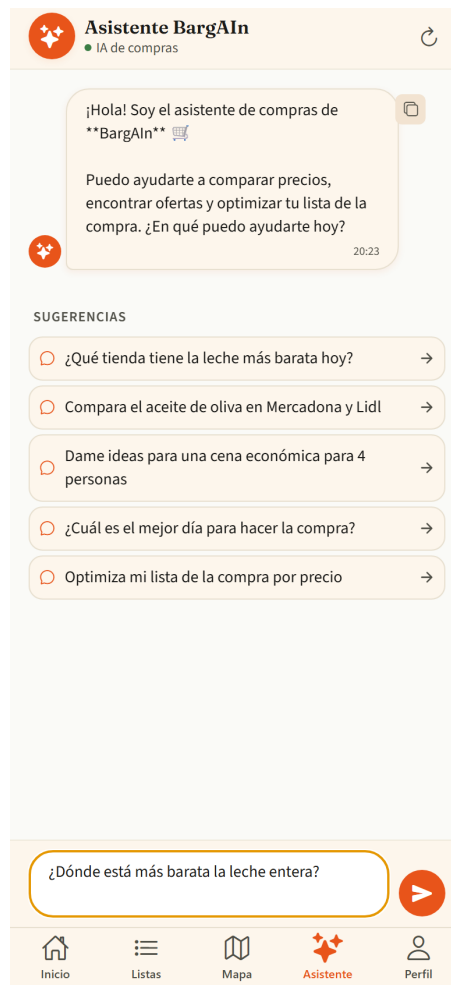


Figura 7.14: Asistente conversacional con sugerencias de inicio y campo de consulta; las respuestas con productos incluyen fichas de añadido directo a la lista

7.13— Lista de comprobación en pantalla de bloqueo (iOS)

Como complemento del flujo de compra presencial, desde la pantalla de ruta optimizada el botón *Checklist en bloqueo* publica una notificación interactiva con los productos de la parada actual. El usuario puede marcar cada artículo como comprado directamente desde la pantalla bloqueada del teléfono, sin desbloquear el dispositivo ni reabrir la aplicación; la notificación se actualiza tras cada acción. La versión basada en *Live Activities* de ActivityKit queda fuera del flujo gestionado de Expo y se documenta como trabajo futuro en el ADR-010.

7.14— La aplicación en el navegador de escritorio

La misma base de código Expo de la aplicación móvil se ejecuta en navegadores de escritorio, donde la interfaz se reorganiza para aprovechar la superficie adicional y los dispositivos de entrada propios del ordenador. Esta sección documenta dicha adaptación, que no es una mera ampliación del lienzo: cambia la disposición, la densidad de información y las *affordances* de interacción.

La pantalla principal (Figura 7.15, izquierda) centra el contenido a un ancho legible y convierte las acciones rápidas en una botonera horizontal. El catálogo (Figura 7.15, derecha) pasa de la lista vertical móvil a una rejilla de dos a cuatro columnas según el ancho de la ventana, con realces *hover* sobre las tarjetas; el atajo **Cmd/Ctrl+K** enfoca el buscador desde cualquier punto, **Enter** añade el producto resaltado a la lista y **Esc** cierra los diálogos abiertos.

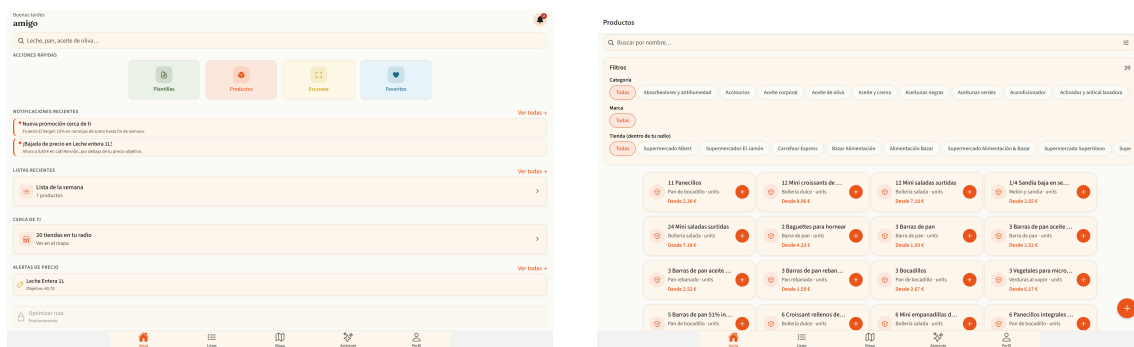


Figura 7.15: Versión de escritorio: pantalla principal con accesos horizontales (izquierda) y catálogo en rejilla multicolumna con soporte de teclado (derecha)

La comparativa de precios (Figura 7.16, izquierda) gana una cabecera fija durante el desplazamiento y un botón de exportación de la tabla a **CSV**, pensado para el usuario que desea elaborar sus propios análisis. En el detalle de lista (Figura 7.16, derecha) aparece una interacción exclusiva del escritorio: el reordenado de ítems mediante arrastre (*drag-and-drop*), válido durante la sesión, junto con la exportación de la lista en texto plano y la copia de un enlace directo de compartición.

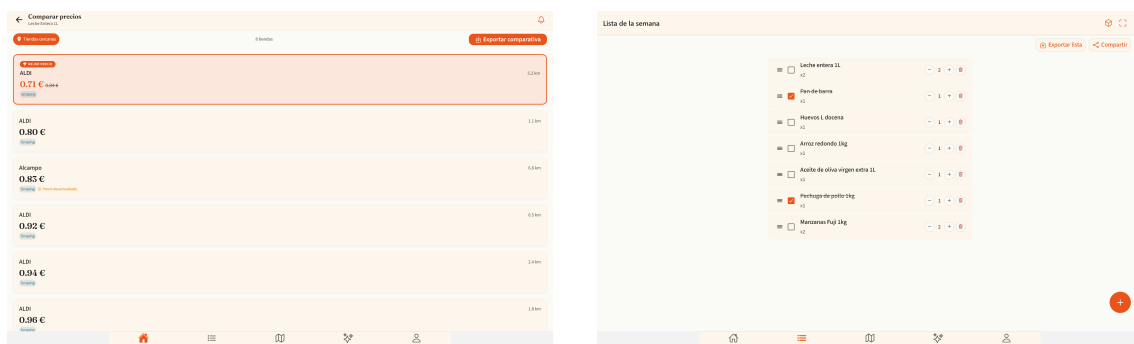


Figura 7.16: Comparativa de precios con cabecera fija y exportación CSV (izquierda) y detalle de lista con reordenado por arrastre y exportación (derecha)

Las vistas de colección se benefician igualmente de la rejilla responsiva: tanto las listas como las plantillas (Figura 7.17) se presentan como tarjetas en múltiples columnas, lo que facilita la comparación visual entre compras recurrentes.

En el mapa (Figura 7.18, izquierda), el panel de tiendas abandona la posición inferior del móvil y se ancla como columna lateral derecha, de forma que mapa y listado per-

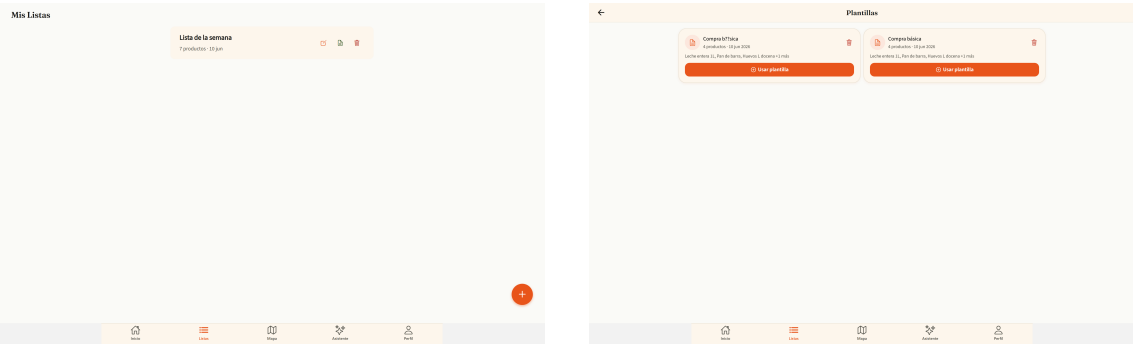


Figura 7.17: Colecciones en rejilla responsiva: listas de la compra (izquierda) y plantillas (derecha)

manecen visibles simultáneamente. El perfil de tienda (Figura 7.18, derecha) adopta una composición a dos columnas —información del establecimiento y catálogo de productos— e incorpora la copia de la dirección al portapapeles.

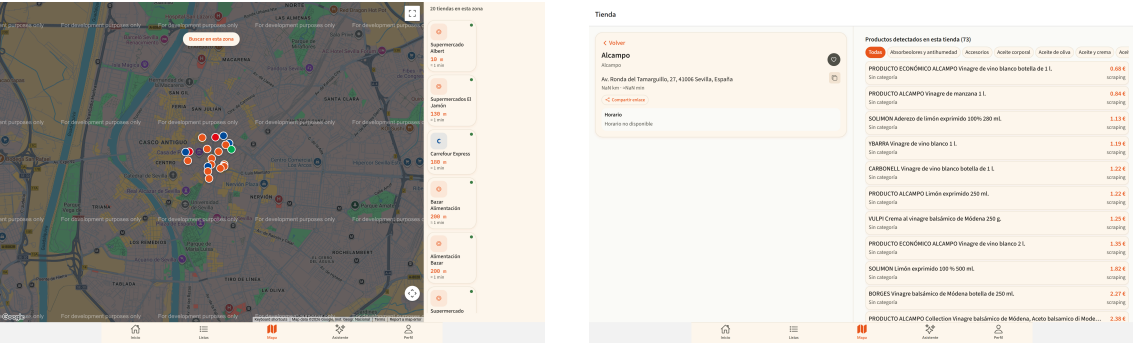


Figura 7.18: Mapa con panel lateral de tiendas (izquierda) y perfil de tienda a dos columnas (derecha)

Las tiendas favoritas se muestran en rejilla (Figura 7.19, izquierda), y el asistente conversacional (Figura 7.19, derecha) centra la conversación a un ancho de lectura cómodo, recibe el foco automáticamente al abrirse y añade la copia individual de cada respuesta.

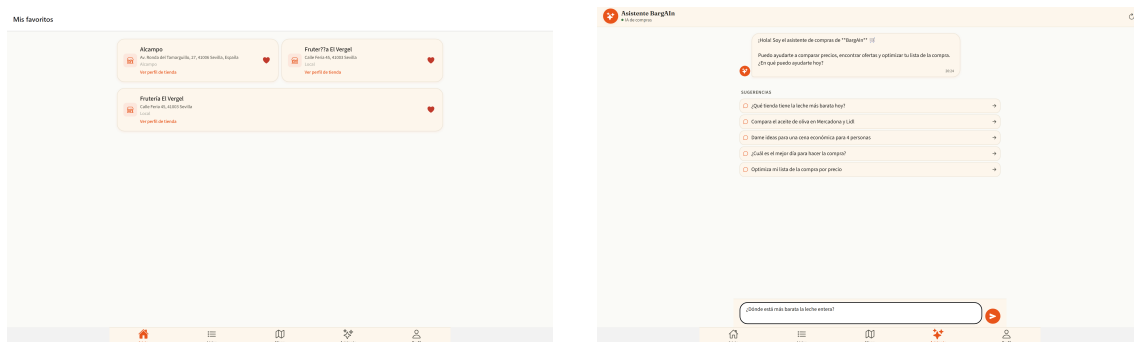


Figura 7.19: Tiendas favoritas en rejilla (izquierda) y asistente conversacional centrado con copia por mensaje (derecha)

La ruta optimizada (Figura 7.20, izquierda) reproduce el desglose por paradas del móvil ampliado con la exportación del itinerario, mientras que el módulo OCR (Figura 7.20, derecha) centra el asistente de captura y delega en el selector de ficheros del sistema cuando no hay cámara disponible.

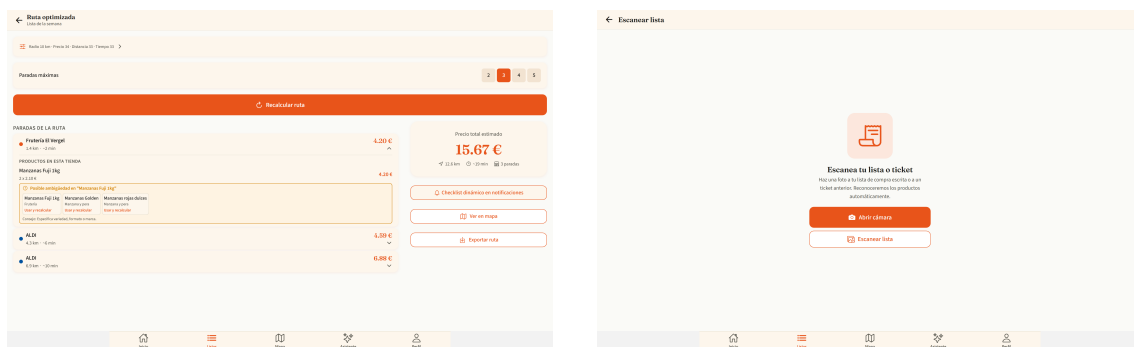


Figura 7.20: Ruta optimizada con exportación del itinerario (izquierda) y módulo OCR adaptado al escritorio (derecha)

Por último, la bandeja de notificaciones y la gestión de alertas (Figura 7.21) se centran a un ancho fijo que preserve la legibilidad de cada aviso. Cada pantalla relevante de la versión web dispone además de URL propia, de modo que cualquier estado —una búsqueda concreta del catálogo mediante `?q=`, una lista, una tienda— puede guardarse como marcador o compartirse como enlace.

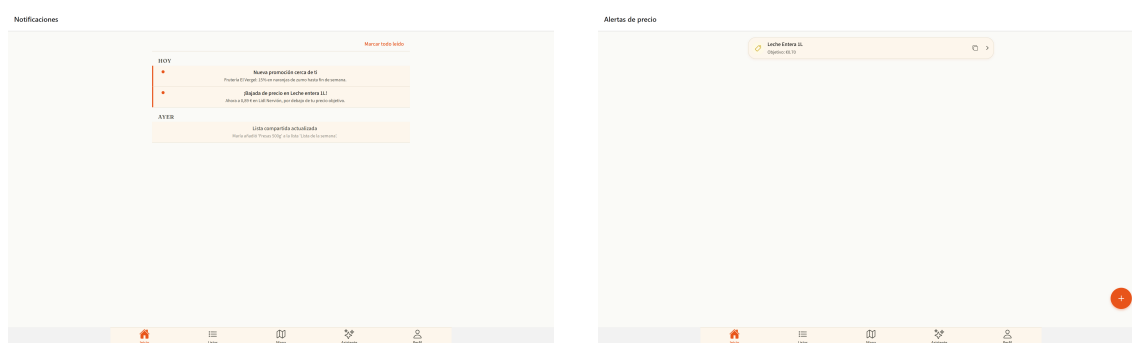


Figura 7.21: Bandeja de notificaciones (izquierda) y alertas de precio (derecha) en la versión de escritorio

7.15– Portal Business para PYMEs

El portal *business* es la contrapartida del sistema para el pequeño comercio: una aplicación web independiente desde la que un establecimiento local publica sus precios y promociones para competir en igualdad de condiciones con las grandes cadenas dentro de BargAIn. Su página de presentación (Figura 7.22) expone la propuesta de valor para el comercio —visibilidad ante consumidores cercanos, gestión autónoma de precios y promociones, y estimaciones orientativas de retorno— y resume el proceso de alta en tres pasos.



Figura 7.22: Página de presentación del portal Business, con la propuesta de valor para el comercio local y el proceso de alta en tres pasos

El acceso al portal (Figura 7.23) reutiliza el mismo sistema de autenticación JWT de la aplicación de consumo, con cuentas de rol *business*. Tras el registro, el comerciante completa el *onboarding* (Figura 7.24): un formulario con los datos legales y comerciales del negocio —razón social, CIF/NIF, dirección y sitio web— que crea el perfil de empresa en estado *pendiente*. Ningún comercio publica precios sin que un administrador haya verificado previamente esos datos, una salvaguarda imprescindible para sostener la confianza del consumidor en la información que muestra la aplicación.

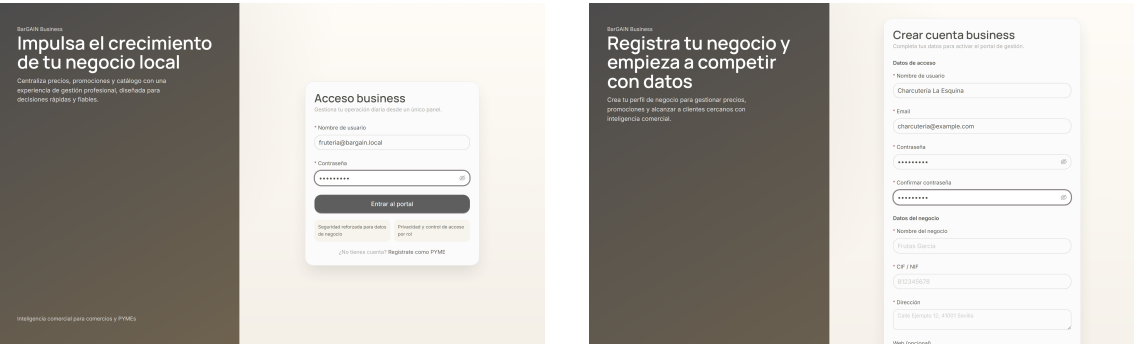


Figura 7.23: Acceso al portal Business: inicio de sesión (izquierda) y registro de una cuenta de comercio (derecha)



Figura 7.24: *Onboarding* del comercio: alta de los datos legales y comerciales que un administrador verificará antes de activar la cuenta

Una vez aprobado, el comerciante accede a su centro de operaciones (Figura 7.25): un panel ejecutivo con los indicadores esenciales del negocio en la plataforma —tiendas activas, precios publicados, promociones vigentes y visualizaciones de perfil acumuladas— , la salud de sus campañas y los últimos ajustes de precio aplicados. El propósito del panel es que un comerciante sin formación analítica pueda responder de un vistazo a las dos preguntas que le importan: qué estoy ofreciendo ahora mismo y cuánta atención está recibiendo.

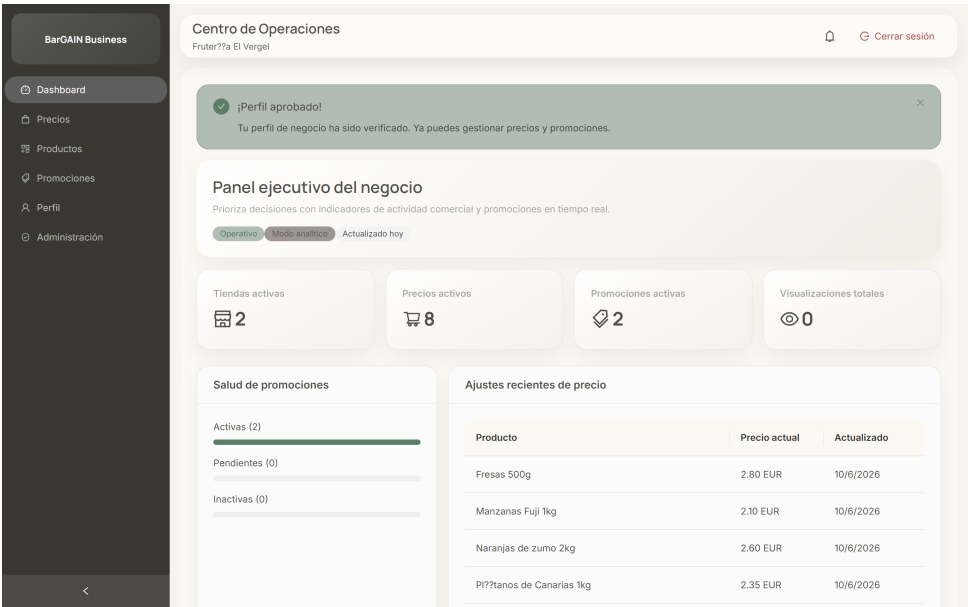


Figura 7.25: Centro de operaciones del comercio: indicadores de actividad, salud de promociones y ajustes recientes de precio

La gestión del surtido se apoya en dos vistas complementarias. La pantalla de productos (Figura 7.26, izquierda) resume el catálogo del comercio con el número de tiendas que ofrecen cada artículo, su precio base mínimo y la mejor oferta vigente. Para incorporaciones masivas, la carga por CSV (Figura 7.26, derecha) permite importar el catálogo completo en una sola operación, con validación fila a fila e informe de incidencias, lo que rebaja considerablemente la barrera de entrada para comercios con decenas de referencias.

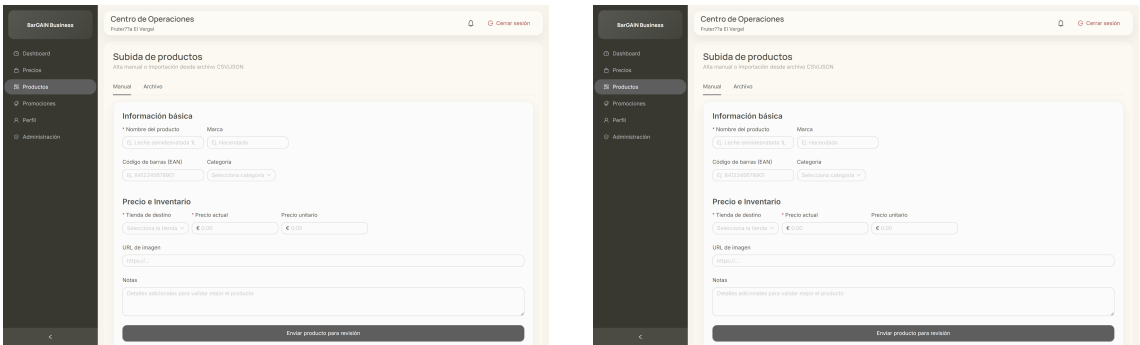


Figura 7.26: Gestión del surtido: resumen de productos del comercio (izquierda) y carga masiva de catálogo mediante CSV con validación (derecha)

El mantenimiento diario de precios se realiza en la tabla editable de la Figura 7.27: cada fila representa un precio publicado (producto–tienda) y el panel lateral permite modificar el precio base, el precio unitario y, opcionalmente, un precio de oferta con fecha de caducidad. Los cambios se publican de forma inmediata en la aplicación de consumo con la fuente *business*, que el sistema de prioridades de BargAIN sitúa por delante de los datos de *scraping* por tratarse de información de primera mano.

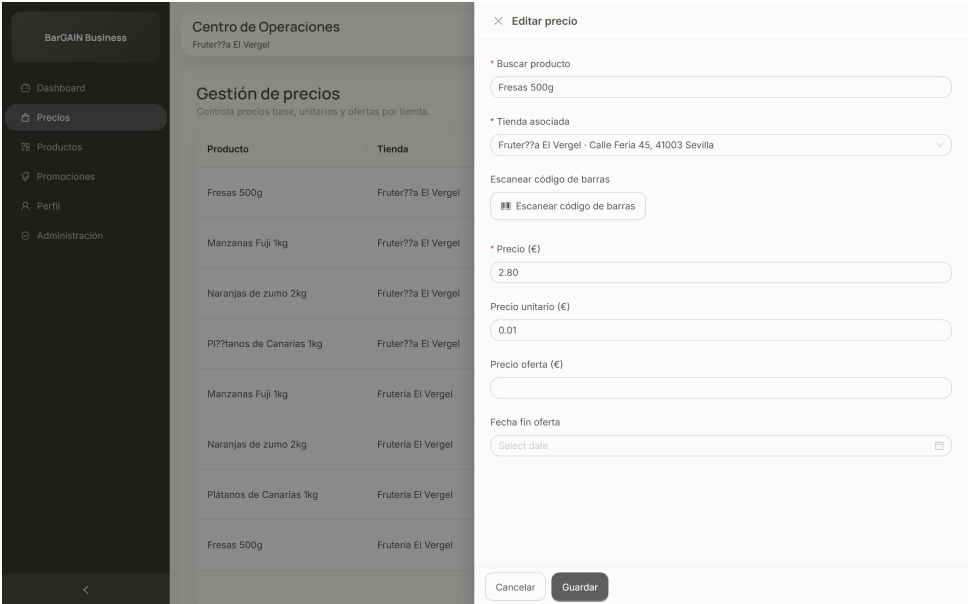


Figura 7.27: Gestión de precios: tabla editable con panel lateral de edición de precio base, unitario y oferta con caducidad

Sobre los precios se construyen las promociones (Figura 7.28, izquierda): descuentos fijos o porcentuales con fecha de inicio y fin, cantidad mínima opcional y título comercial, cuyo rendimiento (visualizaciones, estado) se supervisa desde el propio listado. Las promociones activas se difunden a los consumidores del entorno mediante notificaciones, cerrando el círculo entre la oferta del comercio y la demanda cercana. El perfil del negocio (Figura 7.28, derecha) completa la gestión con los datos públicos del establecimiento y el umbral de alerta competitiva: si un competidor del entorno publica un precio inferior en más del porcentaje configurado, el comerciante recibe un aviso automático.

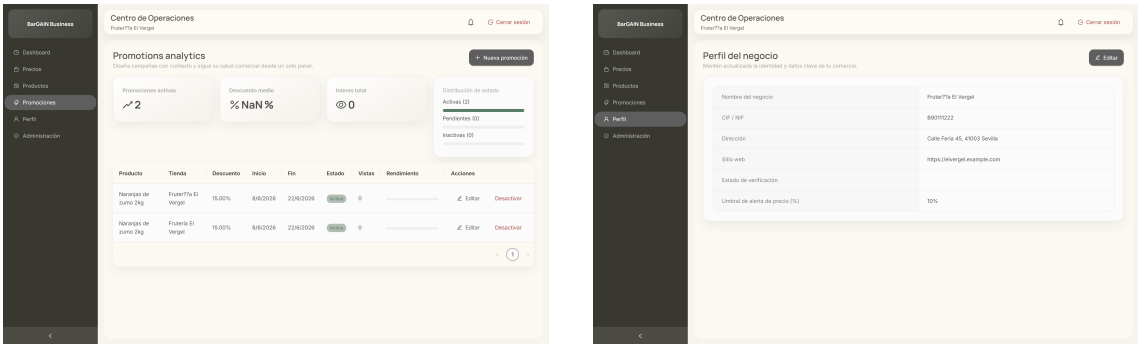


Figura 7.28: Promociones del comercio con analítica básica (izquierda) y perfil del negocio con umbral de alerta competitiva (derecha)

El ciclo administrativo se cierra en el panel de administración (Figura 7.29), reservado a usuarios con privilegios de gestión. En él se revisan los perfiles PYME pendientes —con sus datos legales a la vista— y las propuestas de producto remitidas por los consumidores, con acciones de aprobación y rechazo motivado. Este punto único de moderación garantiza que tanto los comercios como el catálogo crecen de forma controlada.

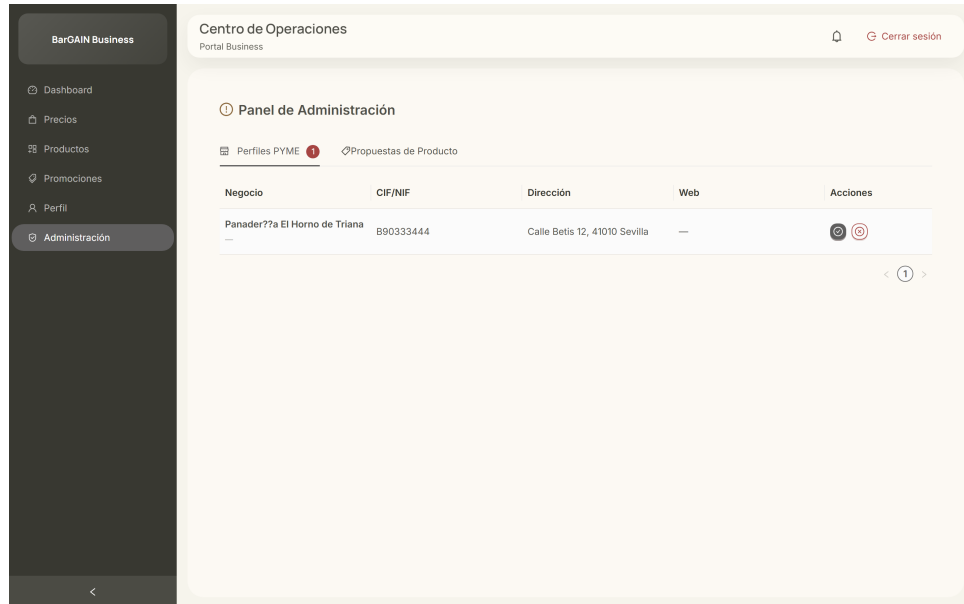


Figura 7.29: Panel de administración: verificación y aprobación de perfiles PYME y de propuestas de producto

7.16— Resolución de incidencias comunes

Se recogen, por último, las incidencias más habituales en el uso del sistema y su resolución:

- **El frontend no conecta con el backend:** verificar que el backend está levantado (`make dev`) y expuesto en `http://localhost:8000`.
- **Errores de autenticación al navegar:** cerrar sesión e iniciarla de nuevo para regenerar el par de tokens.
- **No aparecen tiendas en el mapa:** comprobar los permisos de ubicación del dispositivo o fijar unas coordenadas válidas desde las preferencias del perfil.
- **El OCR no reconoce texto:** procurar buena iluminación y texto legible; la opción de galería con una fotografía de mayor calidad suele resolver el problema.
- **La primera optimización tarda:** tras arrancar el backend, la primera optimización puede demorarse hasta 30 segundos (arranque en frío del servicio de rutas); las siguientes son prácticamente inmediatas.

Pruebas y Resultados

8.1– Objetivo

Validar que BargAIn cumple los requisitos funcionales y no funcionales definidos en la Fase de Análisis, con cobertura automática de los flujos críticos y verificación documentada de los requisitos no funcionales clave (RNF-002, RNF-004, RNF-005).

8.2– Estrategia de testing

Se aplica una estrategia por capas, siguiendo el modelo de pirámide de tests:

- **Tests unitarios backend:** por módulo (`users`, `products`, `stores`, `prices`, `shopping_lists`, `business`, `notifications`, `optimizer`).
- **Tests de integración backend:** flujos cross-domain que ejercitan varios módulos en conjunto.
- **Tests de componentes frontend:** Jest + React Native Testing Library (111 tests en 16 suites), incluyendo los componentes y utilidades de la adaptación web.
- **Tests E2E con Playwright:** 4 flujos críticos del sistema integrado (backend + frontend web).
- **UAT manual en dispositivo móvil:** flujos nativos con GPS y cámara que no son automatizables mediante Playwright (verificados en iPhone con la app compilada).

8.3– Entorno de ejecución

- **Backend:** ejecución dentro del contenedor Docker (modelo híbrido ADR-002). Pytest con configuración en `backend/pytest.ini`, `DJANGO_SETTINGS_MODULE = config.settings.test`.
- **E2E Playwright:** ejecución nativa en host (no Docker) contra el backend Docker local o el staging de Render. Instalado en `frontend/web/` como dependencia de desarrollo.

Los comandos de ejecución principales son:

```
# Backend (desde raiz del repo)
make test-backend          # Tests con -v --tb=short
make test-backend-cov      # Con cobertura HTML

# E2E (desde host)
cd frontend/web && npx playwright test
cd frontend/web && npx playwright test --reporter=html
```

8.4– Suite de tests backend

8.4.1. Tests de integración

Los tests de integración verifican flujos API completos usando una base de datos de test real (sin mocks de BD), validando el comportamiento end-to-end del backend.

Cuadro 8.1: Suite de tests de integración backend

ARCHIVO DE TEST	MÓDULO	DESCRIPCIÓN
test_auth_endpoints.py	users	Registro, login, refresh, perfil
test_optimizer_api.py	optimizer	POST /optimize/ con matriz mock
test_ocr_api.py	ocr	POST /ocr/scan/ con Vision API mock
test_business_verification.py	business	Aprobar / rechazar BusinessProfile
test_business_registration.py	business	Onboarding PYME
test_business_prices.py	business	Gestión de precios PYME
test_bulk_prices.py	business	Actualización masiva CSV
test_proposal_admin.py	business	Aprobar / rechazar propuestas (6 tests)
test_list_endpoints.py	shopping_lists	CRUD listas e ítems
test_cross_domain.py	core	Flujos cross-domain
test_notification_dispatch.py	notifications	Despacho push / email
test_notification_events.py	notifications	Eventos de notificación
test_price_endpoints.py	prices	API de precios
test_product_endpoints.py	products	API de productos
test_store_endpoints.py	stores	API de tiendas
test_assistant_api.py	assistant	Endpoint LLM con mock Gemini
test_promotions.py	business	Gestión de promociones

8.4.2. Cobertura

La suite backend alcanza una cobertura mínima del 80 % sobre los módulos de aplicación (apps/), medida con `pytest-cov`. Los módulos con mayor densidad de tests son `users`, `business` y `optimizer`, que concentran la lógica de negocio principal.

8.5– Tests E2E con Playwright

Se desarrollaron 4 tests E2E automatizados con Playwright para verificar los flujos críticos del sistema integrado (backend + web companion Vite+React):

Los flujos de optimizador y OCR se implementan como tests API-level (usando el fixture `request` de Playwright, sin interfaz de usuario) porque estas funcionalidades son exclusivas de la app móvil — el web companion es un portal business y no dispone de pantallas de consumidor. Este enfoque verifica la integración backend end-to-end de los features principales del TFG sin requerir una interfaz web que no existe.

Cuadro 8.2: Suite de tests E2E con Playwright

FLUJO	ARCHIVO	TIPO	DESCRIPCIÓN
Autenticación completa	auth.spec.ts	UI + API	Registro → login → token refresh → logout
Lista → Optimizar → Ruta	optimizer.spec.ts	API-level	Crear lista → añadir ítems → optimizar → verificar resultado
OCR ticket → lista	ocr.spec.ts	API-level	Subir imagen → endpoint OCR → respuesta estructurada
Business portal	business.spec.ts	UI	Registro PYME → admin aprueba → PYME accede a dashboard

8.5.1. Configuración Playwright

- **Ubicación:** frontend/web/playwright.config.ts
- **Target:** web companion en http://localhost:5173 (o PLAYWRIGHT_BASE_URL)
- **Backend:** Docker local o staging Render (PLAYWRIGHT_API_URL)
- **Navegador:** Chromium (un solo proyecto para evitar duplicación de estado de BD)
- **Workers:** 1 (secuencial — los tests comparten la misma base de datos)

8.6– UAT manual en dispositivo móvil

Los flujos que requieren APIs nativas del dispositivo (GPS, cámara) se verifican manualmente en un iPhone con la app compilada mediante el workflow ios-build.yml:

Cuadro 8.3: Resultados UAT en dispositivo móvil

FLUJO UAT	RESULTADO
Solicitar ubicación y ver tiendas en mapa	Verificado
Crear lista y añadir ítems manualmente	Verificado
Capturar ticket con cámara y reconocer productos	Verificado
Lanzar optimización y navegar la ruta sugerida	Verificado
Chat con asistente LLM y verificar guardrail	Verificado

8.7– Validación de Requisitos No Funcionales

8.7.1. RNF-002: Disponibilidad ≥ 99 % en staging

El backend de staging se desplegó en Render.com (plan gratuito, blueprint render.free.yaml) con la siguiente arquitectura:

- Web Service único: Django con Gunicorn (2 workers) y un proceso Celery worker + beat embebido (-concurrency 1) para las tareas asíncronas.
- PostgreSQL 16 + PostGIS gestionado por la plataforma.
- Redis: broker Celery + caché Google Places.

El health check se configura en GET /api/v1/health/ respondiendo HTTP 200. Los reinicios automáticos ante fallos se gestionan por la plataforma Render. Como optimización del arranque en frío del free tier, los ficheros estáticos se precompilan durante el build de la imagen Docker en lugar de generarse en el arranque del servicio.

8.7.2. RNF-004: Usabilidad WCAG 2.1 AA y máximo 3 taps

Los flujos principales se completan en ≤ 3 interacciones en la app móvil:

- **Ver precios cercanos:** Home → Buscar → Resultados (3 taps)
- **Optimizar ruta:** Lista → Optimizar → Ver ruta (3 taps)
- **Añadir ítem OCR:** Lista → Cámara → Confirmar (3 taps)

8.7.3. RNF-005: Escalabilidad para 10.000 usuarios

La arquitectura de BargAI soporta el crecimiento a 10.000 usuarios concurrentes mediante cuatro decisiones de diseño complementarias:

1. **Workers Celery independientes:** el scraping, el OCR y la optimización se ejecutan en workers separados del API principal, evitando que tareas pesadas bloqueen la gestión de peticiones web.
2. **Redis como broker desacoplado:** las peticiones de usuario se encolan en Redis y se procesan de forma asíncrona, permitiendo absorber picos de carga sin timeout de petición.
3. **PostGIS con índices GiST:** las consultas de tiendas cercanas utilizan índices espaciales con complejidad $O(\log n)$, independiente del volumen total de tiendas registradas.
4. **API stateless con JWT:** el backend no mantiene estado de sesión en servidor. Escalar horizontalmente (añadir réplicas del Web Service) no requiere sincronización de sesiones.

8.8– Resumen de resultados

Cuadro 8.4: Resumen de resultados de la estrategia de testing

TIPO DE TEST	CANTIDAD	ESTADO
Tests unitarios backend	162 (18 archivos)	Todos pasan
Tests de integración backend	171 (17 archivos)	Todos pasan
Tests frontend (Jest)	111 (16 suites)	Todos pasan
Tests E2E Playwright	4 flujos	Todos pasan
UAT manual móvil	5 flujos	Verificados
RNF-002 · Disponibilidad	1	Monitorización en curso
RNF-004 · Usabilidad	3 flujos	Verificados (3 taps máx.)
RNF-005 · Escalabilidad	Justificación arquitectural	Documentado

8.9– Conclusión

La estrategia de testing multicapa garantiza la calidad del sistema a diferentes niveles de granularidad. La cobertura de tests backend supera el 80% sobre los módulos críticos (333 tests entre unitarios y de integración), la suite frontend añade 111 tests de componentes y utilidades con Jest, y los 4 flujos E2E automatizados con Playwright verifican

que el sistema integrado funciona correctamente de extremo a extremo. Los requisitos no funcionales de disponibilidad y usabilidad quedan documentados con evidencia real del entorno de staging.

Conclusiones

9.1— Grado de cumplimiento

El proyecto BargAIn ha completado todos los bloques de desarrollo planificados:

- **F1:** análisis, requisitos y diseño de arquitectura.
- **F2:** infraestructura base de desarrollo y CI/CD.
- **F3:** backend core completo con todos los módulos de dominio y API documentada.
- **F4:** frontend base y avanzado — autenticación, listas, catálogo, mapa, notificaciones, perfil y Places enrichment.
- **F5:** sistema de optimización multicriterio (OR-Tools + OpenRouteService), scraping de precios (Scrapy + Playwright, 4 cadenas), OCR de tickets (Google Cloud Vision API + fuzzy matching), asistente LLM (Google Gemini 2.0 Flash Lite) y wiring de pantallas móviles a endpoints reales.
- **F6:** portal business completo para PYMEs — servicio de aprobación compartido, gestión de precios y promociones, tests de integración, UAT verificada, notificaciones push y email, sincronización documental.
- **F7:** pruebas integrales (unitarias, de integración y E2E), despliegue en staging y cierre documental de la memoria.

Adicionalmente, tras el cierre del plan inicial se incorporó una fase de adaptación de la app de usuario al uso web de escritorio (diseño responsive, atajos de teclado, exportaciones y deep-linking), documentada en los Capítulos 6 y 7.

El sistema está desplegado en staging (Render.com) y los cuatro flujos E2E críticos han sido verificados de forma automatizada con Playwright.

9.2— Aportaciones principales del trabajo

- **Algoritmo de optimización multicriterio propio:** función de scoring ponderada (precio, distancia, tiempo) con OR-Tools para el VRP y OpenRouteService para distancias reales de conducción. Es el núcleo diferencial del TFG.
- **Pipeline de datos de precios automatizado:** cuatro spiders Scrapy para Mercadona, Carrefour, Lidl y DIA, con programación periódica vía Celery Beat y normalización de producto contra catálogo interno.

- **OCR sobre tickets con Google Vision API:** extracción de texto, fuzzy matching contra catálogo y conversión automática a ítems de lista. Elimina la entrada manual.
- **Portal business para PYMEs:** flujo completo de registro, aprobación por administrador, gestión de precios y promociones, con visibilidad para el consumidor final.
- **Asistente LLM con guardrails de dominio:** Gemini 2.0 Flash Lite con restricción temática a consultas de compra, historial de conversación y rate limiting.
- **Arquitectura full-stack coherente:** separación clara entre apps Django, workers Celery independientes, PostGIS para geoespacial y React Native + Expo para iOS/Android desde una única base de código.
- **Verificación automatizada:** suite de tests backend (unitarios + integración, >80 % cobertura), 111 tests de frontend con Jest, tests E2E con Playwright (4 flujos críticos) y UAT manual en iPhone.

9.3— Limitaciones

- **Cobertura de scraping:** el sistema cubre actualmente cuatro cadenas de supermercados (Mercadona, Carrefour, Lidl, DIA). Las cadenas con protección agresiva contra scrapers (Alcampo, Aldi) requieren mantenimiento adicional de los spiders.
- **Caducidad de precios:** los precios de scraping tienen TTL de 48 horas y los de crowdsourcing 24 horas; pueden no reflejar cambios de precio intradiarios.
- **Cold starts en Render free tier:** el plan gratuito de Render puede tener tiempos de arranque de hasta 30 segundos tras inactividad prolongada, lo que afecta a la primera petición tras un período de reposo.
- **Certificado iOS gratuito:** la distribución con Sideloadly y Apple ID gratuito genera certificados con caducidad de 7 días, que requieren re-sideload periódico para el dispositivo de demo.
- **RNF-005 sin load test real:** la justificación de escalabilidad a 10.000 usuarios concurrentes es arquitectural (Celery workers independientes, PostGIS spatial index, API stateless). Un load test completo requeriría infraestructura de pago fuera del alcance del TFG.

9.4— Lecciones aprendidas

- **Modelo híbrido backend Docker / frontend nativo en host:** determinante para la productividad en Windows. Docker volumes en Windows rompen el HMR de Metro/Expo; ejecutar el frontend nativo elimina este problema manteniendo el backend aislado.
- **Contratos API explícitos y documentación viva:** mantener TASKS.md, ADRs y la memoria actualizados durante el desarrollo (no al final) reduce la fricción entre backend y frontend y facilita la incorporación de agentes IA al flujo.

- **OR-Tools frente a algoritmo propio:** usar la librería de optimización combinatoria de Google (producción en empresas logísticas) es preferible a reinventar el VRP desde cero. El valor del TFG está en la integración y la función de scoring, no en reimplementar un solucionador genérico.
- **OCR cloud vs local:** Google Vision API supera a Tesseract en precisión sobre tickets de supermercado reales (texto impreso variable), justificando el coste de la llamada API frente a la alternativa local.
- **Playwright request fixture para flujos mobile-only:** los flujos de lista y optimizador son exclusivos de la app móvil. Cubrirlos con tests API-level (Playwright request fixture) permite automatización completa sin necesitar una interfaz web consumer.

9.5— Trabajo futuro

1. **Publicación en App Store y Google Play:** el proyecto está técnicamente listo para iniciar el proceso de publicación. Requiere cuenta de desarrollador Apple/-Google y revisión de las guías de contenido.
2. **Ampliación de scrapers:** incorporar Alcampo, Aldi, Lidl.es (versión alternativa), El Corte Inglés alimentación y PYMEs locales vía integración API propia.
3. **Precios en tiempo real:** varios supermercados están desplegando APIs semi-públicas o feeds RSS de precios; la arquitectura está preparada para incorporarlos como fuente `api` en el modelo `Price`.
4. **Sistema de reseñas de productos:** los usuarios podrían valorar la calidad/frescura de productos por tienda, añadiendo una dimensión cualitativa al comparador.
5. **Suscripción premium para PYMEs:** el modelo de negocio contempla tres tiers (`free`, `basic`, `premium`) con funcionalidades diferenciadas; la implementación actual es el tier gratuito.
6. **Internacionalización:** la arquitectura soporta múltiples países; la expansión requiere scrapers por mercado y ajuste de la lógica de normalización de cadenas.

9.6— Cierre

BargAIn demuestra que es posible construir, en el marco de un TFG, un sistema full-stack de complejidad real que integra optimización combinatoria, geoespacial, procesamiento de imágenes, generación de lenguaje natural y scraping automatizado bajo una arquitectura coherente y verificada. El proyecto cumple todos los objetivos planteados en la fase de análisis y deja una base técnica sólida para su evolución como producto real.

Referencias

- Applegate, D. L., Bixby, R. E., Chvatal, V., y Cook, W. J. (2006). *The traveling salesman problem: A computational study*. Princeton University Press.
- Brown, T. B., y cols. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Celery Project. (2026). *Celery documentation*. <https://docs.celeryq.dev/>. (Accessed: 2026-04-09)
- Django Software Foundation. (2026). *Django documentation*. <https://docs.djangoproject.com/>. (Accessed: 2026-04-09)
- Encode OSS Ltd. (2026). *Django rest framework documentation*. <https://www.django-rest-framework.org/>. (Accessed: 2026-04-09)
- ETSII Universidad de Sevilla. (2025). *Guía de trabajo fin de grado*.
- Expo Team. (2026). *Expo documentation*. <https://docs.expo.dev/>. (Accessed: 2026-04-09)
- Google Cloud. (2026). *Cloud vision documentation*. <https://cloud.google.com/vision/docs>. (Accessed: 2026-04-09)
- Google OR-Tools Team. (2026). *Or-tools documentation*. <https://developers.google.com/optimization>. (Accessed: 2026-04-09)
- Junta de Andalucía. (2018). *Informe final de la consulta preliminar del mercado de perfiles profesionales en el ámbito informático*.
- Lewis, P., y cols. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Luxen, D., y Vetter, C. (2011). Real-time routing with openstreetmap data. En *Proceedings of the 19th acm sigspatial international conference on advances in geographic information systems*. doi: 10.1145/2093973.2094062
- Microsoft Playwright Team. (2026). *Playwright documentation*. <https://playwright.dev/docs/intro>. (Accessed: 2026-04-09)
- Obe, R. O., y Hsu, L. S. (2021). *Postgis in action* (3rd ed.). Manning Publications.
- PostGIS Project Steering Committee. (2026). *Postgis documentation*. <https://postgis.net/documentation/>. (Accessed: 2026-04-09)
- PostgreSQL Global Development Group. (2026). *Postgresql 16 documentation*. <https://www.postgresql.org/docs/16/>. (Accessed: 2026-04-09)
- React Native Community. (2026). *React native documentation*. <https://reactnative.dev/docs/getting-started>. (Accessed: 2026-04-09)
- Redis Ltd. (2026). *Redis documentation*. <https://redis.io/docs/>. (Accessed: 2026-04-09)
- Render. (2026). *Render documentation*. <https://render.com/docs>. (Accessed: 2026-04-09)
- Scrapy Developers. (2026). *Scrapy documentation*. <https://docs.scrapy.org/>. (Accessed: 2026-04-09)